



GRADO EN INGENIERÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN

Práctica 3: MANETs en INET

Estudiante 1: Ismael Brea Arias

Estudiante 2: Ángel Berreco Kozlan

A Coruña.

Índice general

1	AODV	1
1.1	Escenario en $t = 10$ s – Reenvío del primer y segundo RREQ (AODV-1.1)	1
1.2	Tabla de enrutamiento antes y después del RREQ (AODV-1.2)	4
1.3	Tabla antes y después del primer RREP (AODV-1.3)	6
1.4	Simulación de caída de un nodo en $t = 15$ s (AODV-1.4)	7
1.5	Contenido del RERR en Wireshark (AODV-1.5)	10
1.6	Propagación del RERR (AODV-1.6)	11
1.7	Tabla de enrutamiento antes y después del RERR (AODV-1.7)	13
1.8	Comportamiento de static1 tras recibir RERR (AODV-1.8)	15
2	DSDV	17
2.1	Primer paquete Hello con hopdistance ≥ 3 en $t = 7$ s (DSDV-2.1)	17
2.2	Valor del sequenceNumber (DSDV-2.2)	20
2.3	Modificación del sequenceNumber (DSDV-2.3)	21
2.4	Tabla de enrutamiento antes y después de recibir el Hello (DSDV-2.4)	22
2.5	Caída de un nodo en $t = 15$ s (DSDV-2.5)	28
2.6	Reparación de la ruta entre static1 y static2 (DSDV-2.6)	30
3	AODV vs. DSDV	31
3.1	Primera transmisión en AODV y DSDV (AODVvsDSDV-3.1)	31
3.2	Recepción del datagrama UdpBasicAppData-0 (AODVvsDSDV-3.2)	32
3.3	Paquetes perdidos por la caída del nodo (AODVvsDSDV-3.3)	33
3.4	Porcentaje entregado static1 $\rightarrow$ static2 (AODVvsDSDV-3.4)	36
3.5	Porcentaje de paquetes devueltos static2 $\rightarrow$ static1 (AODVvsDSDV-3.5)	39
4	AODV vs. DSDV: gráficas	40
4.1	Número de paquetes recibidos vs. potencia (ADG-4.1)	40
4.2	Número de paquetes recibidos vs. número de nodos (ADG-4.2)	42

4.3 Número de paquetes recibidos vs. velocidad (ADG-4.3)	43
4.4 Porcentaje de paquetes perdidos vs. helloInterval en DSDV (ADG-4.4)	44

Índice de figuras

1.1	Escenario inicial con AODV con nuestra semilla	1
1.2	Escenario con AODV en t=10 segundos	1
1.3	Logs de los nodos que reenvían el primer RREQ	3
1.4	Disminución del TTL en los mensajes RREQ en AODV	3
1.5	Tabla de enrutamiento de mobile[3] antes de recibir el primer RREQ	4
1.6	Tabla de enrutamiento de mobile[3] después de recibir el primer RREQ	4
1.7	Tabla de enrutamiento de mobile[3] antes de recibir el primer RREP	6
1.8	Tabla de enrutamiento de mobile[3] después de recibir el primer RREP	6
1.9	Escenario en t=15 segundos	7
1.10	Escenario en t=15 segundos con mobile[11] caído	7
1.11	Log de detección de caída de mobile[11] por mobile[1]	7
1.12	Log de envío de RRER broadcast por mobile[1]	8
1.13	Logs de envío del RRER por mobile[1]	8
1.14	Captura del paquete del RRER enviado por mobile[1]	10
1.15	Logs1 de reenvío de RRER	11
1.16	Logs2 de reenvío de RRER	11
1.17	Tabla de enrutamiento de mobile[3] antes del RRER	13
1.18	Tabla de enrutamiento de mobile[3] después del RRER	13
1.19	Captura del primer RREQ enviado por static1	15
1.20	Captura del primer paquete RREQ tras el recibimiento del RREP	15
2.1	Escenario con DSDV t=7 segundos	17
2.2	Log del nodo que envía el primer Hello a partir de t=7 segundos con un hop- distance de al menos 3	17
2.3	Tabla de enrutamiento de mobile[9]	18
2.4	Tabla de enrutamiento de mobile[3] antes del Hello de mobile[9]	22
2.5	Tabla de enrutamiento de mobile[0] antes del Hello de mobile[9]	23

2.6	Tabla de enrutamiento de mobile[12] antes del Hello de mobile[9]	23
2.7	Tabla de enrutamiento de mobile[7] antes del Hello de mobile[9]	24
2.8	Tabla de enrutamiento de mobile[3] después del Hello de mobile[9]	24
2.9	Tabla de enrutamiento de mobile[0] después del Hello de mobile[9]	25
2.10	Tabla de enrutamiento de mobile[12] después del Hello de mobile[9]	25
2.11	Tabla de enrutamiento de mobile[7] antes del Hello de mobile[9]	26
2.12	Escenario inicial antes de t=15 s antes de la caída de mobile[6]	28
2.13	Log del mobile[1] avisando de la caída del mobile[6]	28
2.14	Ruta nueva creada tras la caída del nodo mobile[6]	30
3.1	Paquetes obtenidos sin caída de nodo en static2 con AODV	33
3.2	Paquetes obtenidos con caída de nodo en static2 con AODV	33
3.3	Paquetes obtenidos sin caída de nodo en static2 con DSDV	34
3.4	Paquetes obtenidos con caída de nodo en static2 con DSDV	34
3.5	Paquetes enviados por static1 a static2 en 300 segundos con AODV	36
3.6	Paquetes recibidos por static2 de static1 en 300 segundos con AODV	36
3.7	Paquetes enviados por static1 a static2 en 300 segundos con DSDV	37
3.8	Paquetes recibidos por static2 de static1 en 300 segundos con DSDV	37
4.1	Gráfica de paquetes recibidos en static2 según la potencia	40

Capítulo 1

AODV

Hemos utilizado una semilla con valor de 7117.

1.1. Muestra una captura del escenario en $t = 10$ s. ¿Qué nodos reenvían el primer RREQ enviado por static1? ¿Y el segundo RREQ? ¿Por qué?

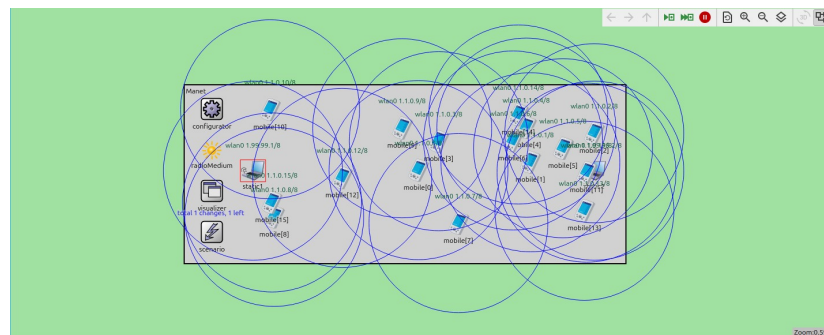


Figura 1.1: Escenario inicial con AODV con nuestra semilla

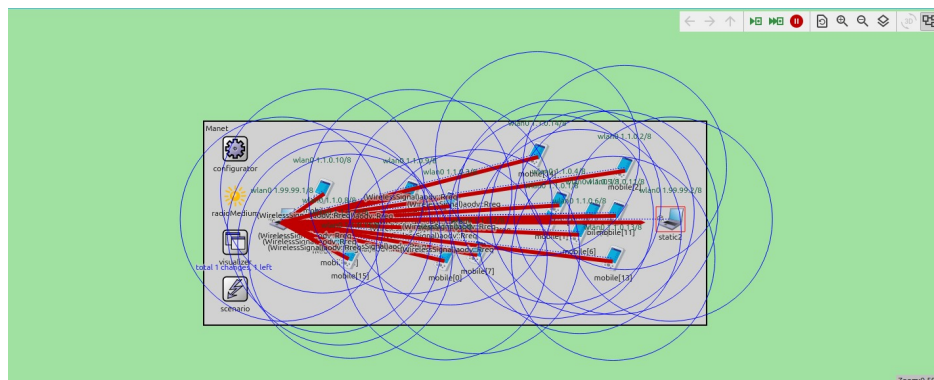


Figura 1.2: Escenario con AODV en $t=10$ segundos

Tal y como se aprecia en el escenario mostrado en la Figura 1.2, el nodo static1 inicia el

descubrimiento de ruta hacia static2 mediante el envío de un primer mensaje RREQ, utilizando el mecanismo de *expanding ring search* definido en el protocolo AODV. Este primer RREQ se envía con un valor de TTL igual a 2. El TTL (Time To Live) es un campo de control que indica el número máximo de saltos que un paquete puede realizar en la red antes de ser descartado. Los nodos que se encuentran más próximos a static1, es decir, los que están dentro de su rango de comunicación, son: mobile[15], mobile[10], mobile[8] y mobile[12]. Al recibir el RREQ de static1, el TTL de este se reduce en 1. Mobile[15] será el primero en reenviar el paquete RREQ, alcanzando a mobile[0] y mobile[9]. Sin embargo, al decrementarse el TTL y este igualarse a 0, ambos nodos móviles no reenviarán el paquete y tan solo almacenarán la información de la ruta inversa por la cual ha circulado dicho paquete. Como consecuencia, el nodo destino static2 no recibe este RREQ y no se genera ningún mensaje RREP. Esta situación puede observarse en la Figura 1.3, donde se muestran los nodos que reenvían el primer RREQ.

Al no recibir respuesta, el nodo origen static1 inicia un segundo intento de descubrimiento de ruta enviando un nuevo RREQ con un TTL mayor, concretamente igual a 4, ampliando así el radio de búsqueda. Este segundo RREQ vuelve a ser recibido inicialmente por los nodos mobile[15], mobile[10], mobile[8] y mobile[12], ahora con un TTL igual a 3. A partir de este punto, el RREQ se propaga progresivamente a nodos más alejados del origen. Mobile[12] lo reenvía a mobile[9] con TTL igual a 2; mobile[9] lo reenvía a mobile[3], mobile[0] y mobile[7] con TTL igual a 1; y finalmente mobile[7] lo reenvía a mobile[1] y mobile[4], mientras que mobile[3] lo reenvía a mobile[14], donde el mensaje llega con TTL igual a 0. En todos estos casos, los nodos que reciben el RREQ actualizan sus tablas de enrutamiento inversas. La propagación progresiva del segundo RREQ y la disminución del TTL en cada salto se ilustran en la Figura 1.4.

Durante esta propagación, el RREQ sigue sin haber alcanzado al nodo destino static2, por lo que el nodo static1 volverá a enviar otro RREQ con un TTL mayor hasta que consiga alcanzar su destino y reciba el RREP del nodo destino.

CAPÍTULO 1. AODV

```
** Event #1830 t=10.002964223082 Manet.static1.wlan[0].radio (Ieee80211ScalarRadio, id=81) on configureRadioMode (inet::Request, id=143317)
INFO: Radio mode changed from RECEIVER to TRANSMITTER.
INFO: Changing radio reception state from IDLE to UNDEFINED.
INFO: Changing radio transmission state from UNDEFINED to IDLE.
** Event #1831 t=10.002964223082 Manet.static1.wlan[0].radio (Ieee80211ScalarRadio, id=81) on aodv::Rreq (inet::Packet, id=143316)
INFO: Transmission started: (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as Ieee80211ScalarTransmitter
INFO: Changing radio transmission state from IDLE to TRANSMITTING.
INFO: Changing radio transmitted signal part from NONE to WHOLE.
** Event #1832 t=10.002964645192 Manet.mobile[6].wlan[0].radio (Ieee80211ScalarRadio, id=742) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143337)
INFO: Reception started: attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReception, pc
INFO: Changing radio reception state from IDLE to RECEIVING.
INFO: Changing radio received signal part from NONE to WHOLE.
** Event #1833 t=10.002964645192 Manet.mobile[10].wlan[0].radio (Ieee80211ScalarRadio, id=874) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143341)
INFO: Reception started: attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReception, pc
INFO: Changing radio reception state from IDLE to RECEIVING.
INFO: Changing radio received signal part from NONE to WHOLE.
** Event #1834 t=10.002964757506 Manet.mobile[12].wlan[0].radio (Ieee80211ScalarRadio, id=1006) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143345)
INFO: Reception started: attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReception, pc
INFO: Changing radio reception state from IDLE to RECEIVING.
INFO: Changing radio received signal part from NONE to WHOLE.
** Event #1835 t=10.002964809599 Manet.mobile[15].wlan[0].radio (Ieee80211ScalarRadio, id=1204) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143351)
INFO: Reception started: attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReception, pc
INFO: Changing radio reception state from IDLE to RECEIVING.
INFO: Changing radio received signal part from NONE to WHOLE.
** Event #1836 t=10.002965347151 Manet.mobile[19].wlan[0].radio (Ieee80211ScalarRadio, id=808) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143359)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1837 t=10.002965663924 Manet.mobile[0].wlan[0].radio (Ieee80211ScalarRadio, id=214) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143321)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1838 t=10.002965673764 Manet.mobile[3].wlan[0].radio (Ieee80211ScalarRadio, id=412) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143327)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1839 t=10.002965927954 Manet.mobile[7].wlan[0].radio (Ieee80211ScalarRadio, id=676) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143325)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1840 t=10.002966486628 Manet.mobile[14].wlan[0].radio (Ieee80211ScalarRadio, id=1138) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143349)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1841 t=10.002966538102 Manet.mobile[1].wlan[0].radio (Ieee80211ScalarRadio, id=288) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143323)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1842 t=10.002966807468 Manet.mobile[4].wlan[0].radio (Ieee80211ScalarRadio, id=478) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143329)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1843 t=10.002966777978 Manet.mobile[6].wlan[0].radio (Ieee80211ScalarRadio, id=610) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143333)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
** Event #1844 t=10.002966808842 Manet.mobile[5].wlan[0].radio (Ieee80211ScalarRadio, id=544) on aodv::Rreq (inet::physicallayer::WirelessSignal, id=143331)
INFO: Reception started: not attempting (inet::physicallayer::WirelessSignal)aodv::Rreq (58 us 99 B) (inet::Packet)aodv::Rreq (99 B) (inet::SequenceChunk) length = 99 B WHOLE as ScalarReceptor
```

Figura 1.3: Logs de los nodos que reenvían el primer RREQ

```
** Event #2256 t=10.32 Manet.static1.routingApp (Aodv, id=72) on selfmsg (inet::aodv::WaitForRrep, id=143312)
INFO: We didn't get any Route Reply within RREP timeout
INFO: Sending a Route Request with target 1.99.99.2 and TTL= 4
** Event #2302 t=10.324119368555 Manet.mobile[12].routingApp (Aodv, id=997) on aodv::Rreq (inet::Packet, id=143832)
INFO: AODV Route Request arrived with source addr: 1.99.99.1 originator addr: 1.99.99.1 destination addr: 1.99.99.2
DETAIL: Updating existing route: destination = 1.99.99.1, prefixLength = 32, nextHop = 1.99.99.1, metric = 1, interface = wlan0
DETAIL: Route updated: destination = 1.99.99.1, prefixLength = 32, nextHop = 1.99.99.1, metric = 1, interface = wlan0
DETAIL: Updating existing route: destination = 1.99.99.1, prefixLength = 32, nextHop = 1.99.99.1, metric = 1, interface = wlan0
DETAIL: Route updated: destination = 1.99.99.1, prefixLength = 32, nextHop = 1.99.99.1, metric = 1, interface = wlan0
INFO: Forwarding the Route Request message with TTL= 3
** Event #2379 t=10.326142040415 Manet.mobile[9].routingApp (Aodv, id=799) on aodv::Rreq (inet::Packet, id=143921)
INFO: AODV Route Request arrived with source addr: 1.1.0.12 originator addr: 1.99.99.1 destination addr: 1.99.99.2
DETAIL: Updating existing route: destination = 1.1.0.12, prefixLength = 32, nextHop = 1.1.0.12, metric = 1, interface = wlan0
DETAIL: Route updated: destination = 1.1.0.12, prefixLength = 32, nextHop = 1.1.0.12, metric = 1, interface = wlan0
DETAIL: Updating existing route: destination = 1.99.99.1, prefixLength = 32, nextHop = 1.1.0.15, metric = 2, interface = wlan0
DETAIL: Route updated: destination = 1.99.99.1, prefixLength = 32, nextHop = 1.1.0.12, metric = 2, interface = wlan0
INFO: Forwarding the Route Request message with TTL= 2
** Event #2460 t=10.326555990978 Manet.mobile[7].routingApp (Aodv, id=667) on aodv::Rreq (inet::Packet, id=144014)
INFO: AODV Route Request arrived with source addr: 1.1.0.9 originator addr: 1.99.99.1 destination addr: 1.99.99.2
DETAIL: Adding new route destination = 1.1.0.9, prefixLength = 32, nextHop = 1.1.0.9, metric = 1, interface = wlan0
INFO (Ipv4RoutingTable)Manet.mobile[7].ipv4.routingTable: add route ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 2, lifetime = 13.326555990978
DETAIL: Adding new route destination = 1.99.99.1, prefixLength = 32, nextHop = 1.1.0.9, metric = 3, interface = wlan0
INFO (Ipv4RoutingTable)Manet.mobile[7].ipv4.routingTable: add route ??? 1.99.99.1/32 gw:1.1.0.9 metric:3 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 2, lifetime = 15.686555990978
INFO: Forwarding the Route Request message with TTL= 1
** Event #2692 t=10.32845589422 Manet.mobile[1].routingApp (Aodv, id=271) on aodv::Rreq (inet::Packet, id=144338)
INFO: AODV Route Request arrived with source addr: 1.1.0.7 originator addr: 1.99.99.1 destination addr: 1.99.99.2
DETAIL: Adding new route destination = 1.1.0.7, prefixLength = 32, nextHop = 1.1.0.7, metric = 1, interface = wlan0
INFO (Ipv4RoutingTable)Manet.mobile[1].ipv4.routingTable: add route ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 2, lifetime = 13.32845589422
DETAIL: Adding new route destination = 1.99.99.1, prefixLength = 32, nextHop = 1.1.0.7, metric = 4, interface = wlan0
INFO (Ipv4RoutingTable)Manet.mobile[1].ipv4.routingTable: add route ??? 1.99.99.1/32 gw:1.1.0.7 metric:4 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 2, lifetime = 15.60845589422
WARN: Can't forward the RREQ because of its small (<= 1) TTL: 0 or the AODV reboot has not completed yet
** Event #2704 t=10.328456008511 Manet.mobile[4].routingApp (Aodv, id=469) on aodv::Rreq (inet::Packet, id=144342)
INFO: AODV Route Request arrived with source addr: 1.1.0.7 originator addr: 1.99.99.1 destination addr: 1.99.99.2
DETAIL: Adding new route destination = 1.1.0.7, prefixLength = 32, nextHop = 1.1.0.7, metric = 1, interface = wlan0
INFO (Ipv4RoutingTable)Manet.mobile[4].ipv4.routingTable: add route ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 2, lifetime = 13.328456008511
DETAIL: Adding new route destination = 1.99.99.1, prefixLength = 32, nextHop = 1.1.0.7, metric = 4, interface = wlan0
INFO (Ipv4RoutingTable)Manet.mobile[4].ipv4.routingTable: add route ??? 1.99.99.1/32 gw:1.1.0.7 metric:4 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 2, lifetime = 15.608456008511
WARN: Can't forward the RREQ because of its small (<= 1) TTL: 0 or the AODV reboot has not completed yet
```

Figura 1.4: Disminución del TTL en los mensajes RREQ en AODV

1.2. Elige un nodo intermedio central de la ruta del primer RREQ que llega a static2 y muestra su tabla de enrutamiento justo antes y justo después de recibir el primer RREQ (rreqId = 1). Explica las diferencias.

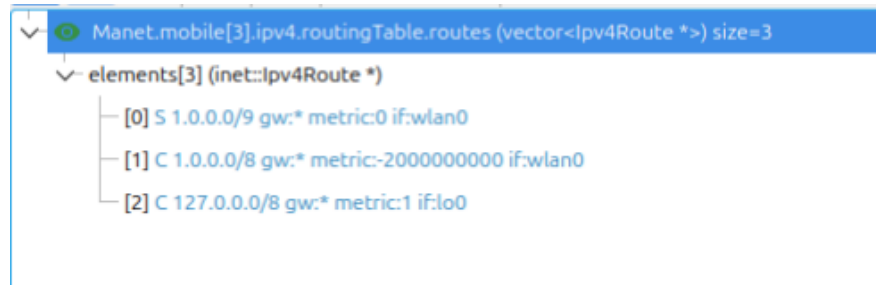


Figura 1.5: Tabla de enrutamiento de mobile[3] antes de recibir el primer RREQ



Figura 1.6: Tabla de enrutamiento de mobile[3] después de recibir el primer RREQ

Antes de recibir el RREQ de mobile[9], el nodo intermedio mobile[3] presenta una tabla de enrutamiento mínima (ver Figura 1.5). Esta consta de solo tres entradas correspondientes a rutas configuradas de forma estática o por defecto: la red local 1.0.0.0/8 y 1.0.0.0/9 sobre la interfaz inalámbrica wlan0, y la interfaz de loopback 127.0.0.0/8. En este estado, el nodo no posee información sobre rutas específicas hacia otros hosts y opera únicamente con su configuración base.

Tras recibir el primer RREQ de mobile[9], la tabla de mobile[3] se actualiza con dos nuevas entradas (ver Figura 1.6). La primera corresponde a la dirección de mobile[9], quien le envía el RREQ, y la segunda es la ruta hacia otro destino registrado durante la propagación, 1.99.99.1/32, aprendida a través de mobile[9]. Ambas entradas se generan al procesar el RREQ y contienen más información que las rutas previas, de acuerdo con los requerimientos de AODV. En las dos, el gateway (gw) se establece como 1.1.0.9, indicando que mobile[9] es el nodo por el cual se debe reenviar el tráfico hacia esos destinos. Además, la entrada hacia mobile[9] tiene una métrica de 1, mostrando que es un vecino directo, mientras que la otra entrada registra una métrica de 3, indicando la distancia en saltos hasta ese destino específico. Los parámetros de control en la tabla, como isActive = 1 y destNum, cumplen funciones

importantes para que la ruta funcione correctamente en AODV. El campo `isActive = 1` indica que la ruta está activa y lista para ser usada; si se volviera 0, significaría que la ruta ya no se considera válida. Por su parte, `destNum` es el número de secuencia del destino, que sirve para determinar cuál es la información más reciente sobre esa ruta y evita que se utilicen rutas antiguas que podrían provocar errores o bucles. Además, cada ruta tiene un tiempo de vida (lifetime) limitado, que en este caso es de aproximadamente 13–15 segundos. Esto significa que la ruta no permanecerá en la tabla para siempre; si no se usa ni se actualiza, desaparecerá automáticamente. Por eso se dice que son rutas “blandas” o temporales: solo existen mientras son necesarias para el descubrimiento de rutas y se actualizan con los nuevos RREQ que lleguen.

1.3. Haz lo mismo justo antes y justo después del primer RREP (destSeqNum = 0).

```

Manet.mobile[3].ipv4.routingTable.routes (vector<Ipv4Route *>) size=8
  elements[8] (inet:Ipv4Route *)
    [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.806383850816
    [1] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.807410258742
    [2] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.804263221531
    [3] ??? 1.1.0.14/32 gw:1.1.0.14 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.80759142704
    [4] ??? 1.99.99.1/32 gw:1.1.0.9 metric:3 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 3, lifetime = 16.164263221531
    [5] S 1.0.0.0/9 gw:* metric:0 if:wlan0
    [6] C 1.0.0.0/8 gw:* metric:2000000000 if:wlan0
    [7] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 1.7: Tabla de enrutamiento de mobile[3] antes de recibir el primer RREP

```

Manet.mobile[3].ipv4.routingTable.routes (vector<Ipv4Route *>) size=9
  elements[9] (inet:Ipv4Route *)
    [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.806383850816
    [1] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.807410258742
    [2] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 3, lifetime = 13.804263221531
    [3] ??? 1.1.0.14/32 gw:1.1.0.14 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 0, lifetime = 13.813468092246, precursor list: 1.1.0.9
    [4] ??? 1.99.99.1/32 gw:1.1.0.9 metric:3 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 3, lifetime = 16.164263221531
    [5] ??? 1.99.99.2/32 gw:1.1.0.14 metric:3 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 0, lifetime = 16.813468092246, precursor list: 1.1.0.9
    [6] S 1.0.0.0/9 gw:* metric:0 if:wlan0
    [7] C 1.0.0.0/8 gw:* metric:2000000000 if:wlan0
    [8] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 1.8: Tabla de enrutamiento de mobile[3] después de recibir el primer RREP

Justo antes de recibir el primer RREP, la tabla de enrutamiento del nodo intermedio mobile[3] contiene únicamente las rutas previamente aprendidas durante la fase de difusión del RREQ, junto con las rutas locales y de configuración. En este estado, el nodo dispone de entradas correspondientes a rutas inversas hacia el origen static1, creadas al recibir el RREQ, pero no existe todavía ninguna ruta válida hacia el nodo destino static2. Estas entradas permiten identificar el siguiente salto hacia el origen, pero no posibilitan aún el encaminamiento de datos hacia el destino final.

Tras recibir el primer RREP, generado por el nodo destino static2 con destSeqNum = 0, el nodo mobile[3] actualiza su tabla de enrutamiento añadiendo una nueva entrada correspondiente a la ruta directa hacia static2. Esta ruta se crea utilizando la información contenida en el RREP, estableciendo como siguiente salto el nodo desde el cual se recibió dicho mensaje. La entrada incluye campos relevantes como el destino (static2), el número de saltos hasta el destino, el número de secuencia del destino, el tiempo de vida de la ruta y la interfaz de salida (wlan0). De este modo, el nodo queda capacitado para reenviar tanto el RREP en dirección al origen como, posteriormente, los paquetes de datos hacia el destino.

Este proceso refleja el funcionamiento de AODV, donde los nodos intermedios crean rutas al destino al recibir un RREP, complementando las rutas inversas del RREQ.

1.4. Añade las líneas para simular la caída de un nodo en $t = 15$ s. Explica qué nodo cae, por qué ese, y cuál es el primer nodo en darse cuenta de la caída. ¿Cómo lo detecta? ¿Notifica la caída? Incluye una captura del log.

```

** .hasStatus = true
*.scenario.script = xml("<scenario><at t='15'>\
<shutdown module='mobile[x]'/></at></scenario>")

```

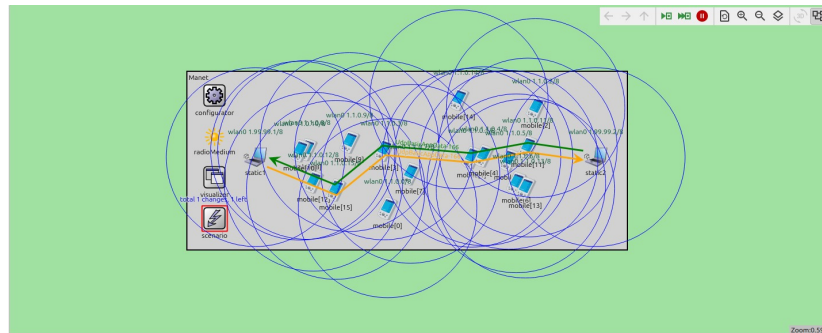


Figura 1.9: Escenario en $t=15$ segundos

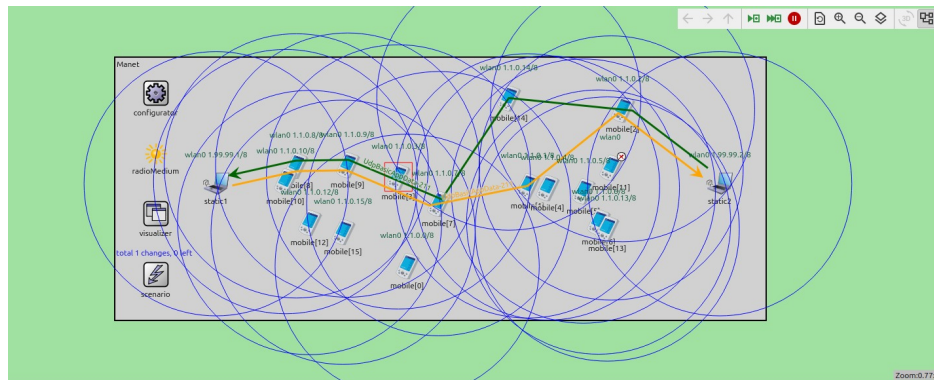


Figura 1.10: Escenario en $t=15$ segundos con mobile[11] caído

```

** Event #231453 1-15.050733319173 Manet.mobile[1].wlan[0].mac.dcf (Dcf, id=285) on selfmsg startRxTimeout (omnetpp::cMessage, id=20873316)
INFO: Frame sequence aborted.
INFO: Data/Mgmt frame transmission failed
INFO: Incremented station SRC's stationShortRetryCounter - 7.
WARN: Retry limit reached for (inet::Packet)UdpBasicAppData-167 (164 B) (inet::SequenceChunk) length = 164 B.
INFO: Dropping frame UdpBasicAppData-167, because retry limit is reached.
DETAIL (Aodv)Manet.mobile[1].routingApp: Received link break signal
DETAIL (Aodv)Manet.mobile[1].routingApp: Marking route to 1.1.0.11 as inactive
DETAIL (Aodv)Manet.mobile[1].routingApp: Marking route to 1.99.99.2 as inactive
INFO (Aodv)Manet.mobile[1].routingApp: Broadcasting Route Error message with TTL=1
INFO (Dcf)Manet.mobile[1].wlan[0].mac.dcf.channelAccess: Channel released.
DETAIL (Contention)Manet.mobile[1].wlan[0].mac.dcf.channelAccess.contention: Starting contention: cw = 31, slots = 20, slotTime = 0.00002, ifs = 0.00005, eifs = 0.000364
INFO (Contention)Manet.mobile[1].wlan[0].mac.dcf.channelAccess.contention: Schedule contention end: backoffSlots = 20, slotTime = 0.00002, lastBusyTime = 15.050732319173, lastIdle = 15.05060

```

Figura 1.11: Log de detección de caída de mobile[11] por mobile[1]

En la Figura 1.10 se observa el escenario inmediatamente después de la caída de moli:

Una vez una decisión al fallo, el primer modo es detectar la muestra del enlace es mobile[1]

Tercero: identificamos `myRegex.matchAll()` inválidamente, automáticamente todas las veces que se

[illegible]

la red.

Como consecuencia de esta notificación, el nodo origen es informado de que la ruta previa ya no es válida y se inicia un nuevo proceso de descubrimiento de ruta. Gracias a ello, el protocolo AODV encuentra una ruta alternativa que no incluye al nodo caído, permitiendo restablecer la comunicación entre static1 y static2. Este comportamiento confirma la capacidad de AODV para adaptarse dinámicamente a cambios en la topología de la red y mantener la conectividad ante fallos de nodos intermedios.

1.5. Muestra el contenido del RERR en Wireshark explicando los campos más importantes. ¿Qué IP tiene como destino? ¿Por qué?

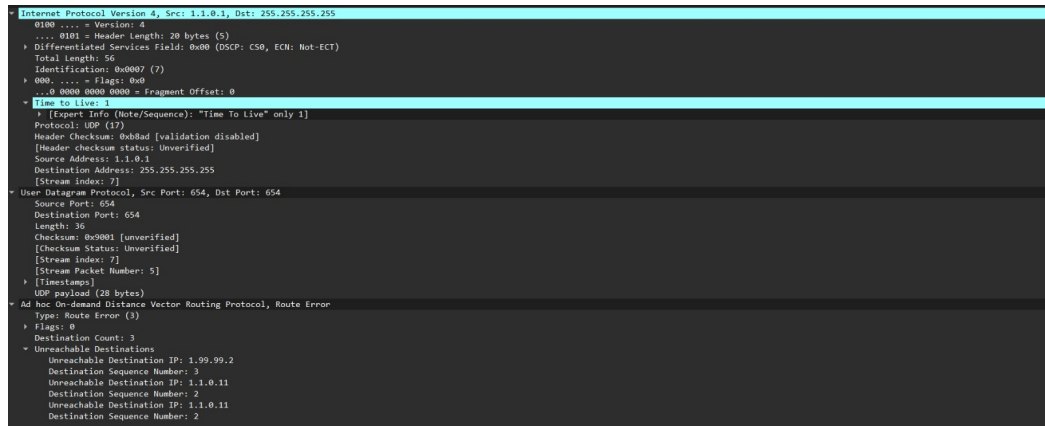


Figura 1.14: Captura del paquete del RERR enviado por mobile[1]

En la captura realizada con Wireshark (Veáse la Figura 1.14) se observa un paquete del protocolo AODV de tipo RERR (Route Error), que se genera tras la caída de un nodo intermedio de la red. El mensaje se transporta sobre UDP, utilizando el puerto 654, que es el puerto estándar empleado por AODV para el intercambio de mensajes de control.

En el encabezado IP del paquete se aprecia que la dirección IP origen es 1.1.0.1, correspondiente al nodo mobile[1], que es el nodo que detecta la pérdida del enlace. La dirección IP de destino es 255.255.255.255, es decir, una dirección de difusión (broadcast). Esto se debe a que los mensajes RERR no se envían a un único nodo concreto, sino que se difunden a todos los vecinos inmediatos para informarles de que ciertas rutas han dejado de ser válidas.

El campo TTL (Time To Live) tiene un valor igual a 1, lo que indica que el mensaje RERR solo debe propagarse un salto. De esta forma, AODV limita la difusión del error únicamente a los nodos directamente afectados, evitando sobrecargar la red con mensajes de control innecesarios.

Dentro del contenido del mensaje AODV se identifica el tipo Route Error (Type = 3) y un Destination Count igual a 3, lo que indica que el mensaje informa de varias rutas que han quedado inalcanzables. Entre los destinos marcados como no alcanzables aparece la IP 1.99.99.2 (nodo static2) y la IP 1.1.0.11, que corresponde al nodo móvil que ha caído. Para cada uno de estos destinos se incluye su número de secuencia, que permite a los nodos receptores actualizar correctamente sus tablas de enrutamiento y descartar rutas obsoletas.

En resumen, el mensaje RERR capturado confirma que, tras la caída del nodo, el protocolo AODV detecta el fallo y notifica a los nodos vecinos mediante un mensaje broadcast, permitiendo que la red invalide las rutas afectadas y pueda iniciar un nuevo proceso de descubrimiento de rutas si es necesario.

1.6. Explica cómo se propaga el RERR por la red. ¿Qué nodos lo reenvían? ¿Cómo sabe un nodo si debe reenviar el RERR?

The screenshot displays a series of log entries from a network simulation. Each entry consists of a timestamp, an IP address, a node identifier, and a log message. The log messages are categorized into 'static' and 'mobile' nodes. The 'static' nodes are represented by 'static::Rerr' and the 'mobile' nodes by 'mobile::Rerr'. The log messages show the propagation of RERR (Route Error) messages across the network. The messages are formatted as 'static::Rerr' or 'mobile::Rerr' followed by the destination IP and the source IP. The log messages show the propagation of RERR (Route Error) messages across the network. The messages are formatted as 'static::Rerr' or 'mobile::Rerr' followed by the destination IP and the source IP.

Figura 1.15: Logs1 de reenvío de RRER

The screenshot displays a series of log entries from a network simulation. Each entry consists of a timestamp, an IP address, a node identifier, and a log message. The log messages are categorized into 'static' and 'mobile' nodes. The 'static' nodes are represented by 'static::Rerr' and the 'mobile' nodes by 'mobile::Rerr'. The log messages show the propagation of RERR (Route Error) messages across the network. The messages are formatted as 'static::Rerr' or 'mobile::Rerr' followed by the destination IP and the source IP. The log messages show the propagation of RERR (Route Error) messages across the network. The messages are formatted as 'static::Rerr' or 'mobile::Rerr' followed by the destination IP and the source IP.

Figura 1.16: Logs2 de reenvío de RRER

Cuando un nodo detecta la caída de un enlace activo, normalmente debido a fallos repetidos en la transmisión a nivel MAC el protocolo AODV genera un mensaje RERR (Route Error) para informar de que determinadas rutas han dejado de ser válidas. Este mensaje se envía en broadcast, con dirección IP 255.255.255.255 y con un TTL igual a 1, lo que limita su propagación a los vecinos directos del nodo que detecta el fallo. Al recibir el RERR, cada nodo comprueba su tabla de enrutamiento y marca como inactivas aquellas rutas cuya siguiente etapa coincide con el nodo caído y cuyos destinos aparecen listados en el mensaje. El reenvío del RERR no es automático: únicamente los nodos que son precursores de alguna de las rutas afectadas lo vuelven a emitir, ya que existen otros nodos que dependen de ellos para alcan-

zar dichos destinos. En nuestro caso, estos son los nodos: mobile[3], mobile[15], mobile[7] y mobile[9]. De este modo, el error se propaga únicamente a lo largo de las rutas implicadas, avisando de forma gradual a los nodos anteriores de la ruta, mientras que los nodos no relacionados con esas rutas descartan el mensaje. Este mecanismo permite una difusión controlada del error, evitando inundar la red y facilitando que AODV inicie, si es necesario, un nuevo proceso de descubrimiento de rutas.

1.7. Muestra capturas de la tabla de enrutamiento de un nodo antes y después de recibir un RERR y explica en qué cambia.

```

Manet.mobile[3].ipv4.routingTable.routes (vector<Ipv4Route *>) size=12
elements[12] (inet:Ipv4Route *)
- [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 15.669137807403
- [1] ??? 1.1.0.1/32 gw:1.1.0.1 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 2, lifetime = 18.070359456536, precursor list: 1.1.0.15
- [2] ??? 1.1.0.5/32 gw:1.1.0.14 metric:2 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 1, lifetime = 16.366347997119
- [3] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 15.673617295298
- [4] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 15.668568938773
- [5] ??? 1.1.0.14/32 gw:1.1.0.14 metric:1 if:wlan0 isActive = 0, hasValidDestNum = 0, destNum = 6, lifetime = 29.739853870421, precursor list: 1.1.0.9
- [6] ??? 1.1.0.15/32 gw:1.1.0.15 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 18.070359456536
- [7] ??? 1.99.99.1/32 gw:1.1.0.15 metric:2 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 9, lifetime = 18.106521172713
- [8] ??? 1.99.99.2/32 gw:1.1.0.1 metric:3 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 2, lifetime = 18.684176080814, precursor list: 1.1.0.9; 1.1.0.15
- [9] S 1.0.0.0/9 gw:* metric:0 if:wlan0
- [10] C 1.0.0.0/8 gw:* metric:-2000000000 if:wlan0
- [11] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 1.17: Tabla de enrutamiento de mobile[3] antes del RRER

```

Manet.mobile[3].ipv4.routingTable.routes (vector<Ipv4Route *>) size=12
elements[12] (inet:Ipv4Route *)
- [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 15.669137807403
- [1] ??? 1.1.0.1/32 gw:1.1.0.1 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 2, lifetime = 18.070359456536, precursor list: 1.1.0.15
- [2] ??? 1.1.0.5/32 gw:1.1.0.14 metric:2 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 1, lifetime = 16.366347997119
- [3] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 15.673617295298
- [4] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 15.668568938773
- [5] ??? 1.1.0.14/32 gw:1.1.0.14 metric:1 if:wlan0 isActive = 0, hasValidDestNum = 0, destNum = 6, lifetime = 29.739853870421, precursor list: 1.1.0.9
- [6] ??? 1.1.0.15/32 gw:1.1.0.15 metric:1 if:wlan0 isActive = 1, hasValidDestNum = 0, destNum = 9, lifetime = 18.070359456536
- [7] ??? 1.99.99.1/32 gw:1.1.0.15 metric:2 if:wlan0 isActive = 1, hasValidDestNum = 1, destNum = 9, lifetime = 18.106521172713
- [8] ??? 1.99.99.2/32 gw:1.1.0.1 metric:3 if:wlan0 isActive = 0, hasValidDestNum = 1, destNum = 3, lifetime = 30.094621179662, precursor list: 1.1.0.9; 1.1.0.15
- [9] S 1.0.0.0/9 gw:* metric:0 if:wlan0
- [10] C 1.0.0.0/8 gw:* metric:-2000000000 if:wlan0
- [11] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 1.18: Tabla de enrutamiento de mobile[3] después del RRER

Antes de recibir el mensaje RERR, la tabla de enrutamiento del nodo mobile[3] (Veáse la Figura 1.17) contiene varias rutas activas hacia distintos destinos, entre ellos static2 (IP 1.99.99.2) y varios nodos móviles intermedios. En estas entradas se observa que el campo `isActive` está a 1 y que existe un next hop válido (por ejemplo `gw: 1.1.0.15` o `gw: 1.1.0.9`), lo que indica que el nodo considera esas rutas utilizables. Además, algunas rutas incluyen una lista de precursores, lo que significa que otros nodos dependen de mobile[3] para alcanzar esos destinos.

Después de que mobile[3] reciba el RERR, la tabla cambia de forma clara (Veáse la Figura 1.18). Las rutas que utilizaban como siguiente salto el nodo que ha caído pasan a marcarse como inactivas, lo que se refleja en el campo `isActive = 0`. En concreto, se observa que la entrada [8], la correspondiente a static2 (1.99.99.2) cambia de estado: aunque la ruta sigue existiendo en la tabla, ya no se considera válida y no puede utilizarse para reenviar paquetes de datos. También se actualiza el número de secuencia del destino, lo que permite evitar que

se reutilice información antigua cuando se descubra una nueva ruta más adelante.

En resumen, tras recibir el RERR el nodo no elimina inmediatamente las rutas afectadas, sino que las invalida, marcándolas como no activas. De esta forma, AODV mantiene la información necesaria para notificar el error a otros nodos (si es precursor) y, al mismo tiempo, impide que se sigan usando rutas rotas. Este cambio en la tabla es lo que desencadena posteriormente un nuevo proceso de descubrimiento de rutas mediante RREQ si el nodo necesita volver a comunicarse con esos destinos.

1.8. ¿Qué hace static1 al recibir el RERR? Muestra el contenido del siguiente RREQ en Wireshark. ¿En qué cambia con respecto al de la pregunta 1?

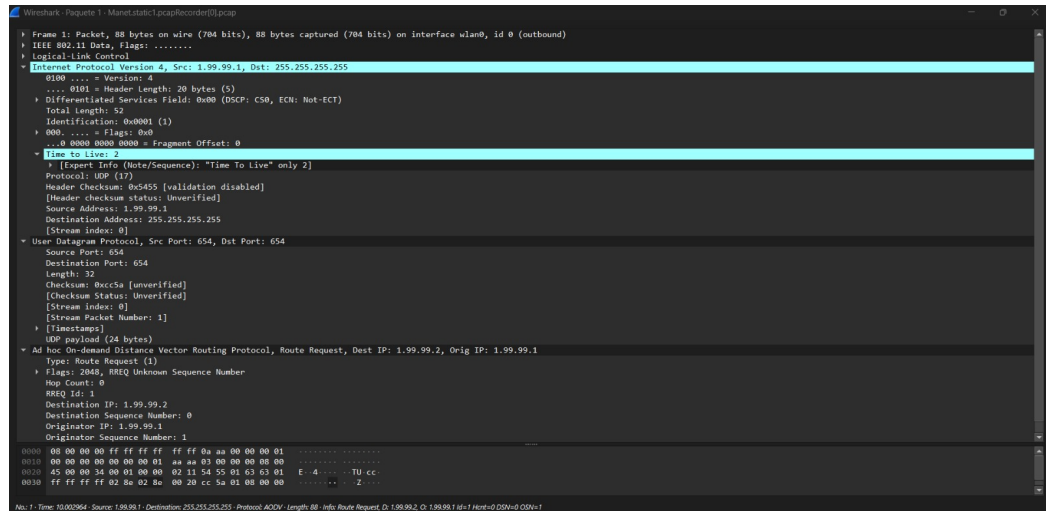


Figura 1.19: Captura del primer RREQ enviado por static1

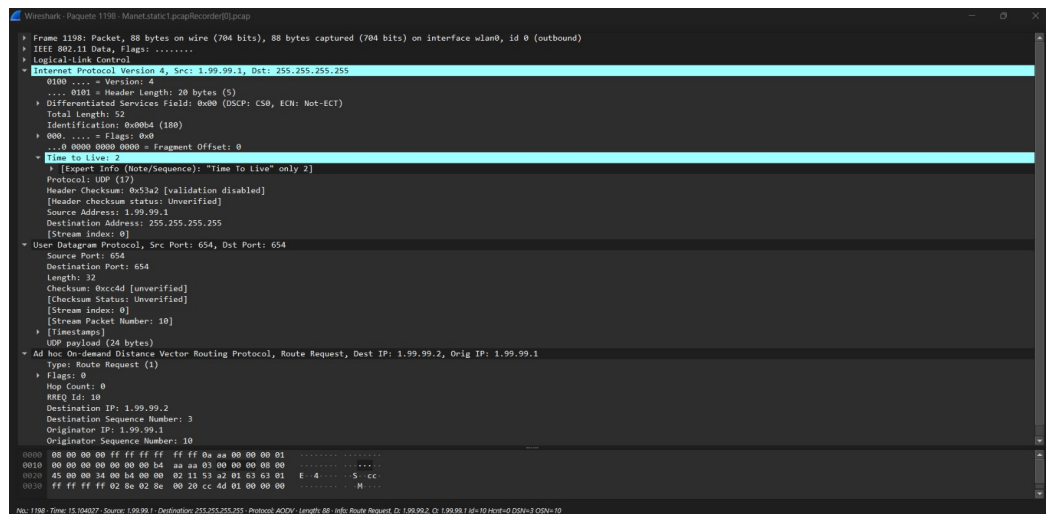


Figura 1.20: Captura del primer paquete RREQ tras el recibimiento del RREP

Cuando static1 recibe el RERR, lo que hace AODV es invalidar la ruta que tenía hacia static2 y reactivar el proceso de descubrimiento de ruta. Esto se ve claramente porque static1 vuelve a emitir un nuevo RREQ en broadcast (destino 255.255.255.255), tal y como aparece en la Figura 1.20. Comparando el RREQ inicial (Veáse la Figura 1.19) con el RREQ enviado tras el RERR (Veáse la Figura 1.20), se observa que el TTL vuelve a ser 2, el Hop Count es 0 y el destino sigue siendo 1.99.99.2, pero cambian los números de secuencia: el RREQ Id y el

Originator Sequence Number son mayores. Esto indica que no es una retransmisión del mismo RREQ, sino una nueva búsqueda de ruta, iniciada porque la anterior ha quedado inválida tras la caída del nodo intermedio. De este modo, static1 fuerza a la red a encontrar una ruta alternativa válida, que será la que finalmente permita restablecer la comunicación con static2.

Capítulo 2

DSDV

2.1. Avanza la simulación hasta el instante $t = 7$ s. Busca el primer paquete Hello transmitido a partir de ese instante con un valor de hopdistance de al menos 3 y muestra una captura del contenido. Explica el significado de los campos `srcAddress` y `nextAddress`, utilizando para explicarlos una captura de la tabla de enrutamiento del nodo que está transmitiendo el paquete (i.e., no el que consta en `srcAddress`).

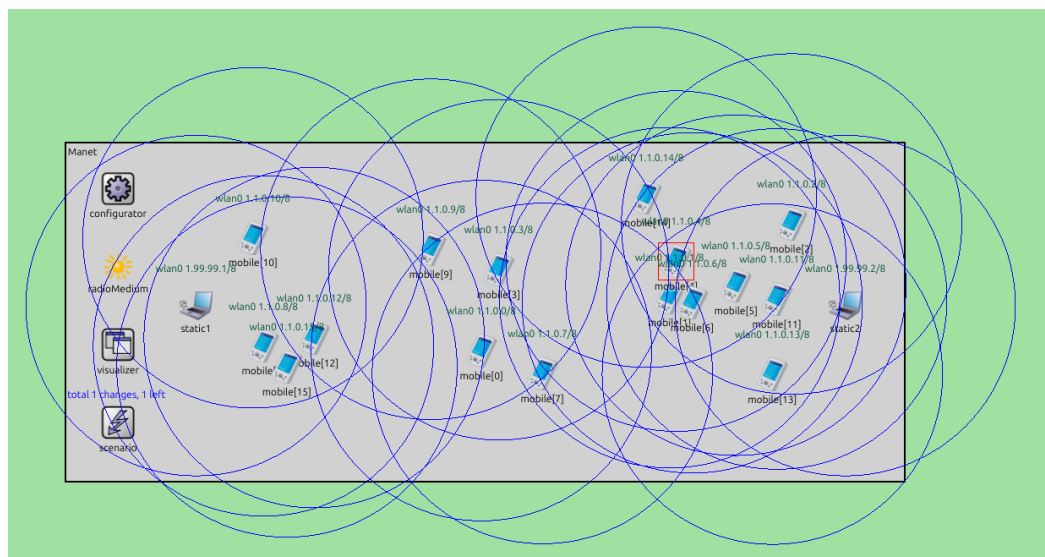


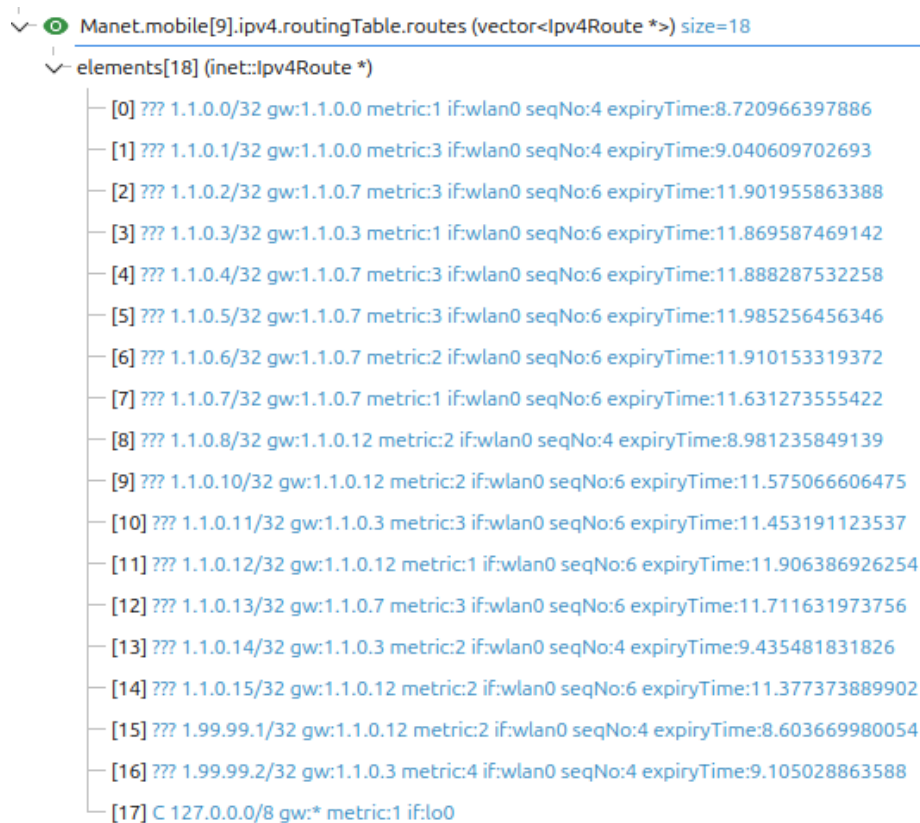
Figura 2.1: Escenario con DSDV $t=7$ segundos

```
** Event #52469 t=7.001631973756 Manet.mobile[9].ipv4.ip (IPv4, id=843) on Hello (inet::Packet, id=56948)
INFO: Received (inet::Packet)Hello (16 B) (inet::DsdvHello) srcAddress = 1.1.0.13, sequencenumber = 6, nextAddress = 1.1.0.9, hopdistance = 4 from upper layer.
DETAIL: Sending datagram 'Hello' with destination = 255.255.255.255
DETAIL: destination address is broadcast, sending packet to broadcast MAC address
INFO: Sending (inet::Packet)Hello (36 B) (inet::SequenceChunk) length = 36 B to output interface = wlan0.
INFO (MessageDispatcher)Manet.mobile[9].ipv4.l3: Dispatching packet to interface, interfaceId = 101, inGate = (omnetpp::cGate)in[1] ← ip.queueOut, outGate = (omnetpp::cGate)in[0]
INFO (MessageDispatcher)Manet.mobile[9].nl: Dispatching packet to interface, interfaceId = 101, inGate = (omnetpp::cGate)in[0] ← ipv4.ifOut, outGate = (omnetpp::cGate)in[1]
INFO (MessageDispatcher)Manet.mobile[9].cb: Dispatching packet to interface, interfaceId = 101, inGate = (omnetpp::cGate)in[0] ← nl.out[1], outGate = (omnetpp::cGate)in[1]
INFO (MessageDispatcher)Manet.mobile[9].bl: Dispatching packet to interface, interfaceId = 101, inGate = (omnetpp::cGate)in[0] ← cb.out[0], outGate = (omnetpp::cGate)in[1]
INFO (MessageDispatcher)Manet.mobile[9].li: Dispatching packet to interface, interfaceId = 101, inGate = (omnetpp::cGate)in[0] ← bl.out[1], outGate = (omnetpp::cGate)in[1]
```

Figura 2.2: Log del nodo que envía el primer Hello a partir de $t=7$ segundos con un hopdistance de al menos 3

En este caso, DSDV, al ser un protocolo proactivo, los nodos mantienen actualizada su tabla de rutas de forma constante mediante la difusión periódica de paquetes de control, incluso antes de que exista tráfico de datos en la red. Como vemos en la Figura 2.2, se avanza la simulación hasta el instante $t=7s$. En el instante 7,001, se identifica el primer paquete Hello capturado en el log del nodo mobile[9], el cual presenta un valor de hopdistance = 4.

De acuerdo con los datos del log capturado, el campo **srcAddress** (1.1.0.13) representa la dirección del nodo que generó originalmente la entrada de ruta (el destino), mientras que el campo **nextAddress** (1.1.0.9) indica cuál es el siguiente salto a través del cual se reenvía la información, es decir, el nodo que transmite el paquete hacia su destino final en ese momento. Este comportamiento se contrasta directamente con la tabla de enrutamiento de mobile[9] (Veáse Figura 2.3), que es el nodo encargado de la transmisión en este evento.



```
Manet.mobile[9].ipv4.routingTable.routes (vector<Ipv4Route *>) size=18
elements[18] (inet::Ipv4Route *)
[0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 seqNo:4 expiryTime:8.720966397886
[1] ??? 1.1.0.1/32 gw:1.1.0.0 metric:3 if:wlan0 seqNo:4 expiryTime:9.040609702693
[2] ??? 1.1.0.2/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.901955863388
[3] ??? 1.1.0.3/32 gw:1.1.0.3 metric:1 if:wlan0 seqNo:6 expiryTime:11.869587469142
[4] ??? 1.1.0.4/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.888287532258
[5] ??? 1.1.0.5/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.985256456346
[6] ??? 1.1.0.6/32 gw:1.1.0.7 metric:2 if:wlan0 seqNo:6 expiryTime:11.910153319372
[7] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 seqNo:6 expiryTime:11.631273555422
[8] ??? 1.1.0.8/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:4 expiryTime:8.981235849139
[9] ??? 1.1.0.10/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:6 expiryTime:11.575066606475
[10] ??? 1.1.0.11/32 gw:1.1.0.3 metric:3 if:wlan0 seqNo:6 expiryTime:11.453191123537
[11] ??? 1.1.0.12/32 gw:1.1.0.12 metric:1 if:wlan0 seqNo:6 expiryTime:11.906386926254
[12] ??? 1.1.0.13/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.711631973756
[13] ??? 1.1.0.14/32 gw:1.1.0.3 metric:2 if:wlan0 seqNo:4 expiryTime:9.435481831826
[14] ??? 1.1.0.15/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:6 expiryTime:11.377373889902
[15] ??? 1.99.99.1/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:4 expiryTime:8.603669980054
[16] ??? 1.99.99.2/32 gw:1.1.0.3 metric:4 if:wlan0 seqNo:4 expiryTime:9.105028863588
[17] C 127.0.0.0/8 gw:* metric:1 if:lo0
```

Figura 2.3: Tabla de enrutamiento de mobile[9]

Al observar la entrada [12] de la tabla de enrutamiento de mobile[9], se comprueba que el destino 1.1.0.13/32 tiene registrada una métrica de 3 y un seqNo: 6. Siguiendo la lógica del protocolo, el nodo mobile[9] toma su métrica interna hacia el destino (3) y le suma 1 antes de propagar el Hello a sus vecinos, resultando en el hopdistance = 4 observado en el log. Esto

indica a los nodos receptores que el destino final se encuentra a 4 saltos si utilizan a mobile[9] como gateway. Además, en la tabla se aprecia que el gateway (gw) para alcanzar al nodo 13 es el 1.1.0.7, lo que demuestra cómo la información fluye desde el lado derecho del escenario hacia la izquierda a través de nodos intermedios.

2.2. ¿Qué valor tiene el *sequenceNumber*? ¿Qué quiere decir ese valor?

El *sequenceNumber* del destino 1.1.0.13, según el log anterior tiene un valor de 6. En DSDV, este número indica la actualidad de la información de la ruta: cuanto mayor es, más reciente es la información conocida sobre ese destino. El valor 6 es par, lo que significa que la ruta hacia el nodo 1.1.0.13 es válida y puede ser utilizada para el reenvío de paquetes.

En otras palabras, el *sequenceNumber* permite que los nodos decidan si deben actualizar sus tablas de enrutamiento al recibir información de otros nodos: solo se reemplaza si el número de secuencia recibido es mayor que el que ya tienen. Además, la paridad del número indica si la ruta es válida (par) o inválida (impar), evitando así bucles o rutas obsoletas en la red.

2.3. ¿Cómo se modifica? ¿Qué nodo lo modifica, y cuándo lo hace?

En el protocolo DSDV, el *sequenceNumber* es gestionado principalmente por el nodo destino (el nodo al que se refiere la ruta). Este número permite que los nodos de la red identifiquen la frescura y validez de la información, decidiendo así si deben actualizar sus tablas de enrutamiento. Los nodos intermedios, por norma general, no modifican este número al retransmitir la ruta; simplemente lo propagan para mantener la integridad de la información.

Cuando la ruta hacia el destino es válida, el nodo destino incrementa su *sequenceNumber* en 2 unidades cada vez que genera una actualización periódica (como un mensaje *Hello*). Este incremento mantiene el número par, indicando a toda la red que el nodo sigue activo y que la información es reciente.

Por otro lado, si un nodo que detecta la caída de un vecino observa que la ruta se ha vuelto inaccesible (por ejemplo, por movimiento o pérdida de enlace), asigna a esa entrada en su tabla un *sequenceNumber* impar (sumando 1 al último número par conocido) y una métrica de infinito. Al propagar este número impar, se notifica rápidamente a la red que la ruta ha caído. El mecanismo de paridad asegura que, en cuanto el nodo destino original reaparezca y emita un nuevo número par (mayor que el impar), la ruta inválida sea sobrescrita automáticamente por la nueva ruta válida, evitando bucles y garantizando la convergencia de la red

2.4. Muestra la tabla de enrutamiento del nodo que recibe el Hello de la pregunta anterior justo antes y justo después de recibirlo, relacionándola con el contenido del paquete. Si se actualiza la tabla, explica por qué se actualiza y las entradas que se crean. Si no se actualiza, explica por qué no se actualiza y di qué entrada se crearía (destino, gateway, métrica) si se actualizase con la información del paquete.

Los nodos que reciben el Hello de mobile[9] son: mobile[3], mobile[0], mobile[12] y mobile[7]. Para ver que cambia comparamos las tablas de enrutamiento antes y después:

-Tablas antes del Hello enviado por mobile[9]:

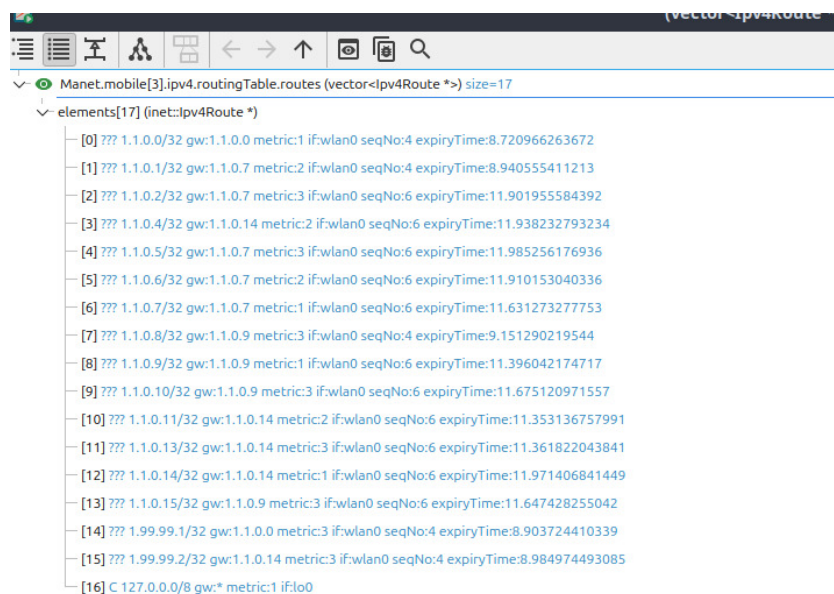


Figura 2.4: Tabla de enrutamiento de mobile[3] antes del Hello de mobile[9]

Manet.mobile[0].ipv4.routingTable.routes (vector<Ipv4Route *>) size=18

elements[18] (inet::Ipv4Route *)

[0]	???	1.1.0.1/32	gw:1.1.0.7	metric:2	if:wlan0	seqNo:4	expiryTime:8.940555182241
[1]	???	1.1.0.2/32	gw:1.1.0.7	metric:3	if:wlan0	seqNo:6	expiryTime:11.901955340306
[2]	???	1.1.0.3/32	gw:1.1.0.3	metric:1	if:wlan0	seqNo:6	expiryTime:11.86958752671
[3]	???	1.1.0.4/32	gw:1.1.0.7	metric:3	if:wlan0	seqNo:6	expiryTime:11.888287009215
[4]	???	1.1.0.5/32	gw:1.1.0.7	metric:3	if:wlan0	seqNo:6	expiryTime:11.985255933049
[5]	???	1.1.0.6/32	gw:1.1.0.7	metric:2	if:wlan0	seqNo:6	expiryTime:11.910152796267
[6]	???	1.1.0.7/32	gw:1.1.0.7	metric:1	if:wlan0	seqNo:6	expiryTime:11.631273033322
[7]	???	1.1.0.8/32	gw:1.1.0.12	metric:2	if:wlan0	seqNo:4	expiryTime:8.981235904186
[8]	???	1.1.0.9/32	gw:1.1.0.9	metric:1	if:wlan0	seqNo:6	expiryTime:11.396042378018
[9]	???	1.1.0.10/32	gw:1.1.0.12	metric:2	if:wlan0	seqNo:6	expiryTime:11.575066708771
[10]	???	1.1.0.11/32	gw:1.1.0.3	metric:3	if:wlan0	seqNo:6	expiryTime:11.453191174034
[11]	???	1.1.0.12/32	gw:1.1.0.12	metric:1	if:wlan0	seqNo:6	expiryTime:11.906387034791
[12]	???	1.1.0.13/32	gw:1.1.0.7	metric:3	if:wlan0	seqNo:6	expiryTime:11.711631451321
[13]	???	1.1.0.14/32	gw:1.1.0.3	metric:2	if:wlan0	seqNo:4	expiryTime:9.435481850432
[14]	???	1.1.0.15/32	gw:1.1.0.12	metric:2	if:wlan0	seqNo:6	expiryTime:11.377373988494
[15]	???	1.99.99.1/32	gw:1.1.0.12	metric:2	if:wlan0	seqNo:4	expiryTime:8.603670028481
[16]	???	1.99.99.2/32	gw:1.1.0.7	metric:3	if:wlan0	seqNo:4	expiryTime:9.114974479931
[17]	C	127.0.0.0/8	gw:*	metric:1	if:lo0		

Figura 2.5: Tabla de enrutamiento de mobile[0] antes del Hello de mobile[9]

Manet.mobile[12].ipv4.routingTable.routes (vector<Ipv4Route *>) size=18

elements[18] (inet::Ipv4Route *)

[0]	???	1.1.0.0/32	gw:1.1.0.0	metric:1	if:wlan0	seqNo:4	expiryTime:8.720966690205
[1]	???	1.1.0.1/32	gw:1.1.0.0	metric:3	if:wlan0	seqNo:4	expiryTime:9.040609993668
[2]	???	1.1.0.2/32	gw:1.1.0.0	metric:4	if:wlan0	seqNo:4	expiryTime:9.061764093209
[3]	???	1.1.0.3/32	gw:1.1.0.0	metric:2	if:wlan0	seqNo:4	expiryTime:9.135093700076
[4]	???	1.1.0.4/32	gw:1.1.0.0	metric:4	if:wlan0	seqNo:4	expiryTime:9.251484849536
[5]	???	1.1.0.5/32	gw:1.1.0.9	metric:4	if:wlan0	seqNo:4	expiryTime:9.853213546275
[6]	???	1.1.0.6/32	gw:1.1.0.9	metric:3	if:wlan0	seqNo:4	expiryTime:8.959503124463
[7]	???	1.1.0.7/32	gw:1.1.0.0	metric:2	if:wlan0	seqNo:6	expiryTime:11.911327893745
[8]	???	1.1.0.8/32	gw:1.1.0.8	metric:1	if:wlan0	seqNo:6	expiryTime:11.653929119071
[9]	???	1.1.0.9/32	gw:1.1.0.9	metric:1	if:wlan0	seqNo:6	expiryTime:11.396042561461
[10]	???	1.1.0.10/32	gw:1.1.0.10	metric:1	if:wlan0	seqNo:6	expiryTime:11.445011854281
[11]	???	1.1.0.11/32	gw:1.1.0.9	metric:4	if:wlan0	seqNo:6	expiryTime:11.573296010161
[12]	???	1.1.0.13/32	gw:1.1.0.0	metric:5	if:wlan0	seqNo:6	expiryTime:11.931931321832
[13]	???	1.1.0.14/32	gw:1.1.0.0	metric:3	if:wlan0	seqNo:4	expiryTime:9.47553666903
[14]	???	1.1.0.15/32	gw:1.1.0.15	metric:1	if:wlan0	seqNo:6	expiryTime:11.27731913747
[15]	???	1.99.99.1/32	gw:1.99.99.1	metric:1	if:wlan0	seqNo:6	expiryTime:11.556859169727
[16]	???	1.99.99.2/32	gw:1.1.0.0	metric:4	if:wlan0	seqNo:4	expiryTime:9.535029299517
[17]	C	127.0.0.0/8	gw:*	metric:1	if:lo0		

Figura 2.6: Tabla de enrutamiento de mobile[12] antes del Hello de mobile[9]

```

Manet.mobile[7].ipv4.routingTable.routes (vector<Ipv4Route *>) size=17
  elements[17] (inet:Ipv4Route *)
    [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 seqNo:4 expiryTime:8.720966348021
    [1] ??? 1.1.0.1/32 gw:1.1.0.1 metric:1 if:wlan0 seqNo:6 expiryTime:11.850803015445
    [2] ??? 1.1.0.2/32 gw:1.1.0.1 metric:2 if:wlan0 seqNo:6 expiryTime:11.651901002517
    [3] ??? 1.1.0.3/32 gw:1.1.0.3 metric:1 if:wlan0 seqNo:6 expiryTime:11.869587687422
    [4] ??? 1.1.0.4/32 gw:1.1.0.6 metric:2 if:wlan0 seqNo:6 expiryTime:11.728232670998
    [5] ??? 1.1.0.5/32 gw:1.1.0.1 metric:2 if:wlan0 seqNo:6 expiryTime:11.965201597839
    [6] ??? 1.1.0.6/32 gw:1.1.0.6 metric:1 if:wlan0 seqNo:6 expiryTime:11.430098458734
    [7] ??? 1.1.0.8/32 gw:1.1.0.0 metric:3 if:wlan0 seqNo:4 expiryTime:9.26129034388
    [8] ??? 1.1.0.9/32 gw:1.1.0.3 metric:2 if:wlan0 seqNo:6 expiryTime:11.576096767502
    [9] ??? 1.1.0.10/32 gw:1.1.0.9 metric:3 if:wlan0 seqNo:6 expiryTime:11.675121473819
    [10] ??? 1.1.0.11/32 gw:1.1.0.1 metric:2 if:wlan0 seqNo:6 expiryTime:11.423136387695
    [11] ??? 1.1.0.13/32 gw:1.1.0.1 metric:2 if:wlan0 seqNo:6 expiryTime:11.301577107457
    [12] ??? 1.1.0.14/32 gw:1.1.0.6 metric:2 if:wlan0 seqNo:4 expiryTime:9.335481925432
    [13] ??? 1.1.0.15/32 gw:1.1.0.9 metric:3 if:wlan0 seqNo:6 expiryTime:11.64742875804
    [14] ??? 1.99.99.1/32 gw:1.1.0.0 metric:3 if:wlan0 seqNo:4 expiryTime:8.903724484187
    [15] ??? 1.99.99.2/32 gw:1.1.0.6 metric:2 if:wlan0 seqNo:6 expiryTime:11.913779700886
    [16] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 2.7: Tabla de enrutamiento de mobile[7] antes del Hello de mobile[9]

-Tablas después del Hello enviado por mobile[9]:

```

Manet.mobile[3].ipv4.routingTable.routes (vector<Ipv4Route *>) size=17
  elements[17] (inet:Ipv4Route *)
    [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 seqNo:4 expiryTime:8.720966263672
    [1] ??? 1.1.0.1/32 gw:1.1.0.7 metric:2 if:wlan0 seqNo:4 expiryTime:8.940555411213
    [2] ??? 1.1.0.2/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.901955584392
    [3] ??? 1.1.0.4/32 gw:1.1.0.14 metric:2 if:wlan0 seqNo:6 expiryTime:11.938232793234
    [4] ??? 1.1.0.5/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.985256176936
    [5] ??? 1.1.0.6/32 gw:1.1.0.7 metric:2 if:wlan0 seqNo:6 expiryTime:11.910153040336
    [6] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 seqNo:6 expiryTime:11.631273277753
    [7] ??? 1.1.0.8/32 gw:1.1.0.9 metric:3 if:wlan0 seqNo:4 expiryTime:9.151290219544
    [8] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 seqNo:6 expiryTime:11.396042174717
    [9] ??? 1.1.0.10/32 gw:1.1.0.9 metric:3 if:wlan0 seqNo:6 expiryTime:11.675120971557
    [10] ??? 1.1.0.11/32 gw:1.1.0.14 metric:2 if:wlan0 seqNo:6 expiryTime:11.353136757991
    [11] ??? 1.1.0.13/32 gw:1.1.0.14 metric:3 if:wlan0 seqNo:6 expiryTime:11.361822043841
    [12] ??? 1.1.0.14/32 gw:1.1.0.14 metric:1 if:wlan0 seqNo:6 expiryTime:11.971406841449
    [13] ??? 1.1.0.15/32 gw:1.1.0.9 metric:3 if:wlan0 seqNo:6 expiryTime:11.647428255042
    [14] ??? 1.99.99.1/32 gw:1.1.0.0 metric:3 if:wlan0 seqNo:4 expiryTime:8.903724410339
    [15] ??? 1.99.99.2/32 gw:1.1.0.14 metric:3 if:wlan0 seqNo:4 expiryTime:8.984974493085
    [16] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 2.8: Tabla de enrutamiento de mobile[3] después del Hello de mobile[9]

```

Manet.mobile[0].ipv4.routingTable.routes (vector<Ipv4Route *>) size=18
└─ elements[18] (inet::Ipv4Route *)
  [0] ??? 1.1.0.1/32 gw:1.1.0.7 metric:2 if:wlan0 seqNo:4 expiryTime:8.940555182241
  [1] ??? 1.1.0.2/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.901955340306
  [2] ??? 1.1.0.3/32 gw:1.1.0.3 metric:1 if:wlan0 seqNo:6 expiryTime:11.86958752671
  [3] ??? 1.1.0.4/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.888287009215
  [4] ??? 1.1.0.5/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.985255933049
  [5] ??? 1.1.0.6/32 gw:1.1.0.7 metric:2 if:wlan0 seqNo:6 expiryTime:11.910152796267
  [6] ??? 1.1.0.7/32 gw:1.1.0.7 metric:1 if:wlan0 seqNo:6 expiryTime:11.631273033322
  [7] ??? 1.1.0.8/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:4 expiryTime:8.981235904186
  [8] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 seqNo:6 expiryTime:11.396042378018
  [9] ??? 1.1.0.10/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:6 expiryTime:11.575066708771
  [10] ??? 1.1.0.11/32 gw:1.1.0.3 metric:3 if:wlan0 seqNo:6 expiryTime:11.453191174034
  [11] ??? 1.1.0.12/32 gw:1.1.0.12 metric:1 if:wlan0 seqNo:6 expiryTime:11.906387034791
  [12] ??? 1.1.0.13/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:6 expiryTime:11.711631451321
  [13] ??? 1.1.0.14/32 gw:1.1.0.3 metric:2 if:wlan0 seqNo:4 expiryTime:9.435481850432
  [14] ??? 1.1.0.15/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:6 expiryTime:11.377373988494
  [15] ??? 1.99.99.1/32 gw:1.1.0.12 metric:2 if:wlan0 seqNo:4 expiryTime:8.603670028481
  [16] ??? 1.99.99.2/32 gw:1.1.0.7 metric:3 if:wlan0 seqNo:4 expiryTime:9.114974479931
  [17] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 2.9: Tabla de enrutamiento de mobile[0] después del Hello de mobile[9]

```

Manet.mobile[12].ipv4.routingTable.routes (vector<Ipv4Route *>) size=18
└─ elements[18] (inet::Ipv4Route *)
  [0] ??? 1.1.0.0/32 gw:1.1.0.0 metric:1 if:wlan0 seqNo:4 expiryTime:8.720966690205
  [1] ??? 1.1.0.1/32 gw:1.1.0.0 metric:3 if:wlan0 seqNo:4 expiryTime:9.040609993668
  [2] ??? 1.1.0.2/32 gw:1.1.0.0 metric:4 if:wlan0 seqNo:4 expiryTime:9.061764093209
  [3] ??? 1.1.0.3/32 gw:1.1.0.0 metric:2 if:wlan0 seqNo:4 expiryTime:9.135093700076
  [4] ??? 1.1.0.4/32 gw:1.1.0.0 metric:4 if:wlan0 seqNo:4 expiryTime:9.251484849536
  [5] ??? 1.1.0.5/32 gw:1.1.0.9 metric:4 if:wlan0 seqNo:4 expiryTime:9.853213546275
  [6] ??? 1.1.0.6/32 gw:1.1.0.9 metric:3 if:wlan0 seqNo:4 expiryTime:8.959503124463
  [7] ??? 1.1.0.7/32 gw:1.1.0.0 metric:2 if:wlan0 seqNo:6 expiryTime:11.911327893745
  [8] ??? 1.1.0.8/32 gw:1.1.0.8 metric:1 if:wlan0 seqNo:6 expiryTime:11.653929119071
  [9] ??? 1.1.0.9/32 gw:1.1.0.9 metric:1 if:wlan0 seqNo:6 expiryTime:11.396042561461
  [10] ??? 1.1.0.10/32 gw:1.1.0.10 metric:1 if:wlan0 seqNo:6 expiryTime:11.445011854281
  [11] ??? 1.1.0.11/32 gw:1.1.0.9 metric:4 if:wlan0 seqNo:6 expiryTime:11.573296010161
  [12] ??? 1.1.0.13/32 gw:1.1.0.9 metric:4 if:wlan0 seqNo:6 expiryTime:12.00168672544
  [13] ??? 1.1.0.14/32 gw:1.1.0.0 metric:3 if:wlan0 seqNo:4 expiryTime:9.47553666903
  [14] ??? 1.1.0.15/32 gw:1.1.0.15 metric:1 if:wlan0 seqNo:6 expiryTime:11.27731913747
  [15] ??? 1.99.99.1/32 gw:1.99.99.1 metric:1 if:wlan0 seqNo:6 expiryTime:11.556859169727
  [16] ??? 1.99.99.2/32 gw:1.1.0.0 metric:4 if:wlan0 seqNo:4 expiryTime:9.535029299517
  [17] C 127.0.0.0/8 gw:* metric:1 if:lo0

```

Figura 2.10: Tabla de enrutamiento de mobile[12] después del Hello de mobile[9]

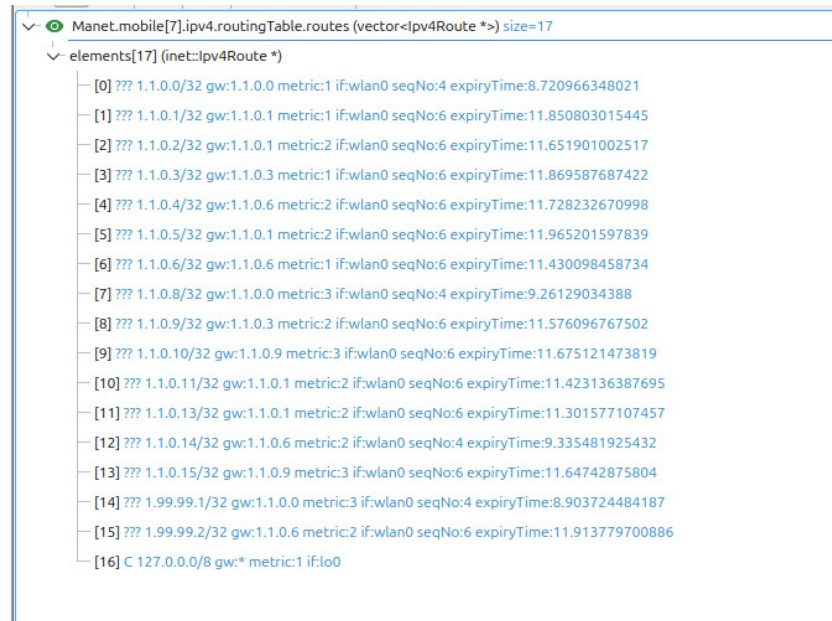


Figura 2.11: Tabla de enrutamiento de mobile[7] antes del Hello de mobile[9]

Como podemos ver, ninguna de las tablas de enrutamiento de los nodos mobile[0], mobile[3], mobile[7] y mobile[12] cambia tras recibir el mensaje Hello enviado por el nodo mobile[9]. Esto ocurre porque la información incluida en el Hello no mejora la información de enrutamiento que estos nodos ya tenían almacenada.

En el mensaje Hello, el nodo mobile[9] anuncia entradas de su tabla de enrutamiento con un sequence number igual al que ya poseen los nodos receptores para dichos destinos. De acuerdo con el funcionamiento del protocolo DSDV, cuando el número de secuencia recibido no es mayor que el almacenado, la entrada no se actualiza, ya que no se considera información más reciente.

Además, en las tablas de enrutamiento de mobile[0], mobile[3], mobile[7] y mobile[12] se observa que las métricas (hop distance) asociadas a las rutas existentes son menores o iguales que las que se obtendrían a través del nodo mobile[8]. Por este motivo, tampoco se actualizan las métricas ni se modifica el gateway, ya que el camino anunciado por mobile[9] no supone una mejora respecto a las rutas ya conocidas.

Por tanto, el Hello recibido únicamente sirve para confirmar la conectividad y mantener viva la información existente, pero no provoca ninguna modificación en la tabla de enrutamiento. En el caso de que alguno de los nodos hubiese recibido un sequence number mayor y par, se habría actualizado la entrada correspondiente, ajustando la métrica y estableciendo como gateway al nodo mobile[9] si el camino fuese mejor. Por otro lado, si el sequence number recibido hubiese sido mayor e impar, la entrada se habría marcado como inalcanzable, indicando la caída de la ruta.

En resumen, los mensajes Hello enviados por mobile[8] cumplen una función de mantenimiento del protocolo, pero no generan cambios en las tablas de enrutamiento de mobile[0], mobile[3], mobile[7] y mobile[12] al no aportar información más reciente ni rutas de menor coste

2.5. Avanza hasta la caída del nodo en $t = 15$ s. Ten en cuenta que la ruta en ese momento puede ser diferente a la de AODV, y por lo tanto el nodo a desactivar también. ¿Cuál es el primer nodo en darse cuenta de la caída? ¿Notifica la caída del nodo de alguna forma?

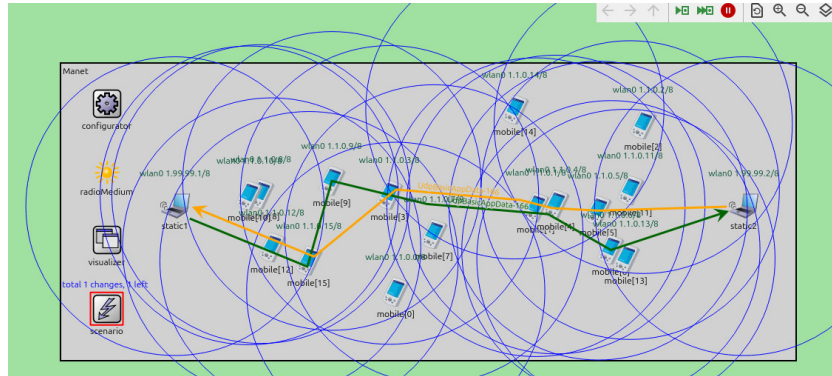


Figura 2.12: Escenario inicial antes de $t=15$ s antes de la caída de mobile[6]

```

** Event #263619 t=15.042963545504 Manet.mobile[1].wlan[0].mac.dcf (Dcf, id=285) on selfmsg startRxTimeout (omnetpp::cMessage, id=825500)
INFO: Frame sequence aborted.
INFO: Data/Mgmt frame transmission failed
INFO: Incremented station SRC: stationShortRetryCounter = 7.
WARN: Retry limit reached for (inet::Packet)UdpBasicAppData-167 (164 B) (inet::SequenceChunk) length = 164 B.
INFO: Dropping frame UdpBasicAppData-167, because retry limit is reached.
INFO (Dcaf)Manet.mobile[1].wlan[0].mac.dcf.channelAccess: Channel released.
DETAIL (Contention)Manet.mobile[1].wlan[0].mac.dcf.channelAccess.contention: Starting contention: cw = 31, slots = 28, slotTime = 0.00002, ifs = 0.00005, eifs = 0.0
INFO (Contention)Manet.mobile[1].wlan[0].mac.dcf.channelAccess.contention: Scheduling contention end: backoffslots = 28, slotTime = 0.00002, lastBusyTime = 15.04295

```

Figura 2.13: Log del mobile[1] avisando de la caída del mobile[6]

En nuestro caso, el nodo que primero se da cuenta de la caída es el mobile[1]. Esto ocurre porque mobile[1] es un nodo directamente implicado en la ruta activa y está intentando transmitir paquetes de datos a través del nodo 6, que es el que se desactiva en $t=15$ s. En el momento en que el nodo 6 deja de estar operativo, mobile[1] continúa intentando enviar tramas MAC hacia él, tal y como se observa en el log, donde aparecen varios intentos fallidos de transmisión y el incremento del contador de reintentos (stationShortRetryCounter). Tras varios intentos sin recibir confirmación, se alcanza el límite de reintentos, lo que provoca que la capa MAC aborte definitivamente la transmisión y libere el canal.

Este comportamiento es precisamente el mecanismo mediante el cual mobile[1] detecta la caída del vecino: no recibe ACKs a nivel MAC y, por tanto, concluye que el enlace con el nodo 6 ya no es válido. En DSDV, esta detección no se notifica mediante un mensaje explícito como ocurre con el RERR en AODV. En su lugar, el nodo que detecta el fallo marca la ruta como inválida en su tabla de enrutamiento, asignándole una métrica infinita y un sequence number impar, indicando al resto de la red que ese destino ya no es alcanzable por ese camino.

La notificación de la caída, por tanto, no se hace de forma inmediata ni puntual, sino que se propaga de manera implícita a través de las siguientes actualizaciones periódicas (mensajes

Hello/actualizaciones DSDV). Cuando mobile[1] envía su siguiente anuncio de enrutamiento, el resto de nodos reciben esa información y actualizan sus tablas en consecuencia, evitando seguir utilizando una ruta que pasa por el nodo 6. De esta forma, la red converge progresivamente hacia una nueva topología válida sin necesidad de un mensaje de error dedicado.

2.6. ¿Cómo se repara la ruta entre static1 y static2? ¿En qué momento?

Cuando cae el nodo intermedio, la ruta entre static1 y static2 se rompe. En DSDV no se busca una nueva ruta de forma inmediata, sino que el nodo que detecta el fallo marca la ruta como inalcanzable (métrica infinita y sequence number impar) y esta información se va propagando al resto de nodos mediante actualizaciones periódicas. La ruta se repara cuando la red vuelve a converger y los nodos reciben anuncios más recientes con sequence numbers pares que indican un nuevo camino válido hacia static2. Cada nodo actualiza entonces su tabla eligiendo la ruta más reciente y con menor métrica. Por ello, la recuperación es gradual y depende del tiempo de propagación de las actualizaciones DSDV, siendo más lenta que en AODV, pero sin usar mensajes reactivos adicionales. En la figura podemos ver como sería la nueva ruta:

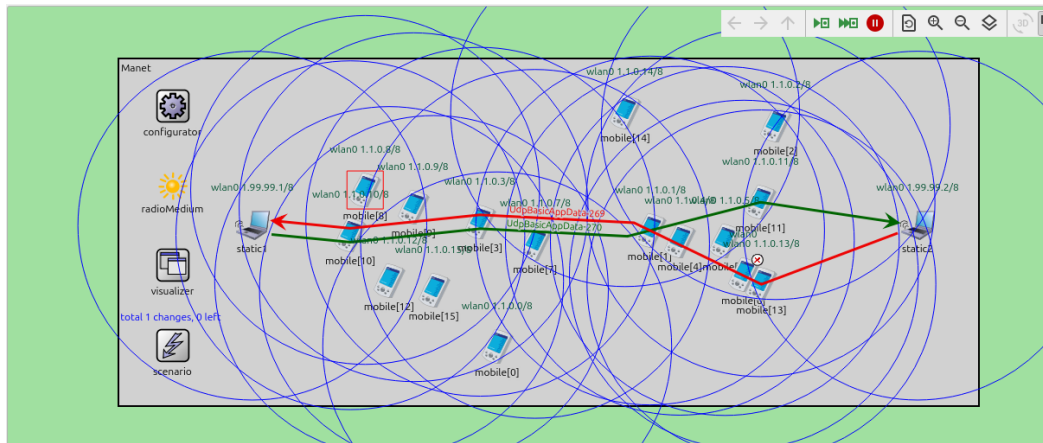


Figura 2.14: Ruta nueva creada tras la caída del nodo mobile[6]

La ruta se repara en el momento en que los nodos comienzan a recibir anuncios DSDV con sequence numbers pares y más recientes que indican un nuevo camino válido hacia static2. Este momento corresponde al instante en el que la red vuelve a converger tras la propagación de las actualizaciones periódicas, que en nuestro caso ha sido en 18,3 segundos aproximadamente.

AODV vs. DSDV

3.1. ¿En qué instante se realiza la primera transmisión (de cualquier tipo de paquete) con AODV? ¿Y con DSDV? ¿Por qué?

En la configuración AODV, la primera transmisión ocurre en el instante $t=10.002964$ s. En este momento, el nodo static1 difunde un paquete de control de tipo RREQ (Route Request) a través de su interfaz inalámbrica hacia los nodos vecinos. El motivo de este instante específico es la naturaleza reactiva del protocolo AODV. A diferencia de los protocolos proactivos, AODV no genera tráfico de enrutamiento de forma periódica, por lo que la red permanece en silencio hasta que una aplicación requiere enviar datos. Dado que la aplicación UdpBasicApp en static1 está configurada para iniciar en $t=10$ s, es en ese segundo cuando el protocolo detecta la falta de una ruta hacia static2 e inicia el proceso de descubrimiento. Este retraso de unos milisegundos tras el inicio de la aplicación ($t=10$) se debe al procesamiento interno de la pila de protocolos y al jitter (pequeño retardo aleatorio) que introduce el protocolo para evitar colisiones al difundir el mensaje de control por la red.

Por el contrario, en la configuración DSDV, la primera transmisión se registra mucho antes, concretamente en el instante $t=0.056712$ s. En este caso, el nodo mobile[13] emite un paquete de control de tipo DsdvHello para anunciar su presencia y estado a los nodos adyacentes. Esta temprana actividad responde a la naturaleza proactiva del protocolo. DSDV no espera a que exista una demanda de tráfico de datos para actuar; en su lugar, los nodos intercambian periódicamente sus tablas de rutas para mantener una visión actualizada de la topología de la red desde el inicio de la simulación. De este modo, cuando la aplicación comienza a transmitir en el segundo 10, las rutas ya han sido establecidas previamente de forma automática.

3.2. ¿En qué instante recibe static2 el datagrama UdpBasicAppData-0 con AODV? ¿Y con DSDV? ¿Por qué?

Con AODV, el nodo static2 recibe el datagrama UdpBasicAppData-0 en el instante $t = 10.821710$ s. Este retardo se debe a que AODV es un protocolo reactivo, por lo que antes de poder entregar el paquete de datos es necesario realizar un proceso de descubrimiento de ruta mediante mensajes de control (RREQ y RREP).

Con DSDV, la recepción del datagrama por static2 se produce antes, en el instante $t = 10.029260$ s, ya que DSDV es un protocolo proactivo y mantiene las rutas actualizadas de forma periódica, permitiendo el envío inmediato de datos sin necesidad de descubrimiento previo.

3.3. ¿Cuántos paquetes se pierden como consecuencia de la caída del nodo en AODV? ¿Y en DSDV? ¿A qué se debe la diferencia?

Para ver esto con claridad en ambos casos hasta los 30 segundos hemos cambiado la semilla para cada caso. Para AODV hemos utilizado una semilla de 71177 y para DSDV hemos utilizado una de 777.

Para AODV sin caída de nodo se han obtenido 483 paquetes recibidos hasta el segundo 30 de simulación. En el caso de caída de nodo (en este caso el nodo mobile[7]), se han recibido 480 paquetes en el mismo intervalo de tiempo. Esto supone una pérdida de 3 paquetes, lo que equivale aproximadamente a un 0,62% de paquetes perdidos respecto al escenario sin caída.

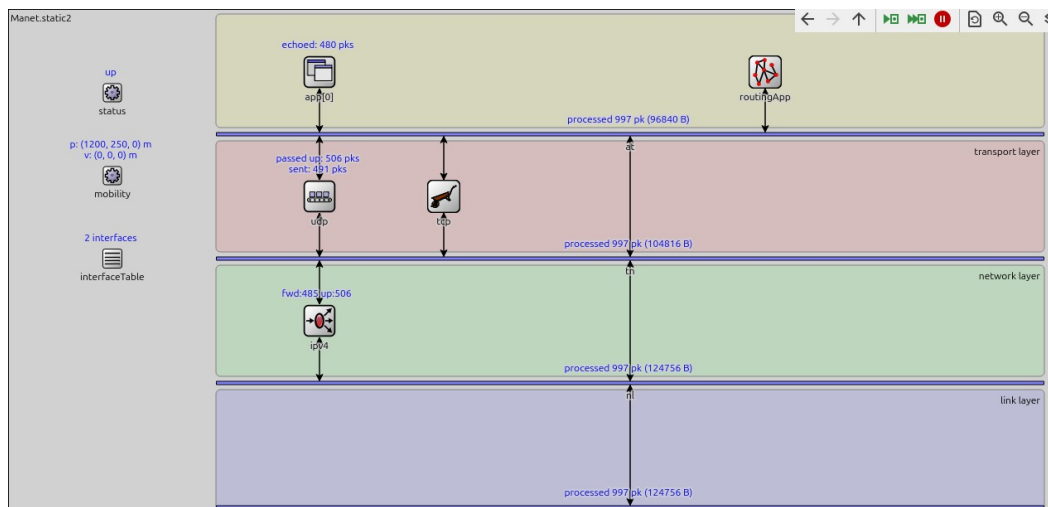


Figura 3.1: Paquetes obtenidos sin caída de nodo en static2 con AODV

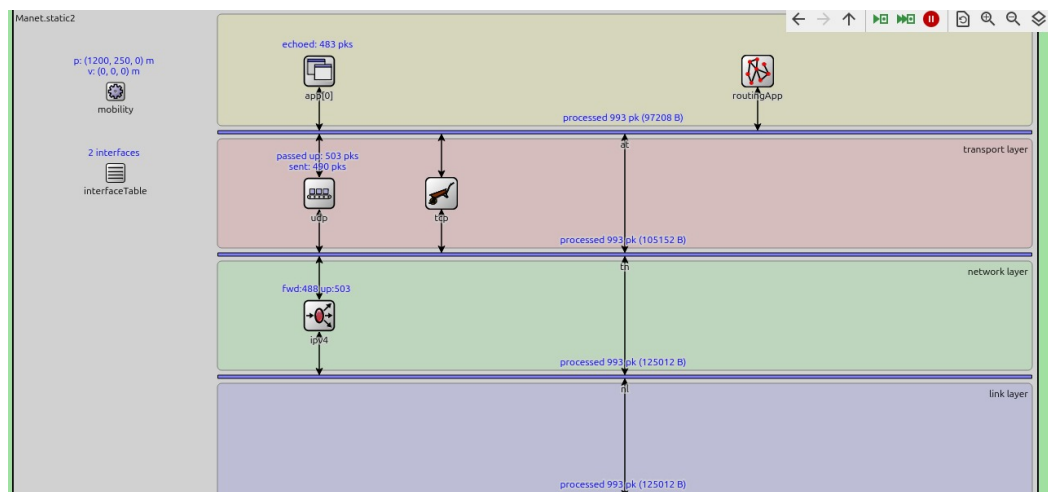


Figura 3.2: Paquetes obtenidos con caída de nodo en static2 con AODV

Para DSDV sin caída de nodo se han obtenido 463 paquetes recibidos hasta el segundo 30 de simulación. En el caso de caída de nodo (en este caso el nodo mobile[9]), se han recibido 319 paquetes en el mismo intervalo de tiempo. Esto supone una pérdida de 144 paquetes, lo que equivale aproximadamente a un 31,1% de paquetes perdidos respecto al escenario sin caída.

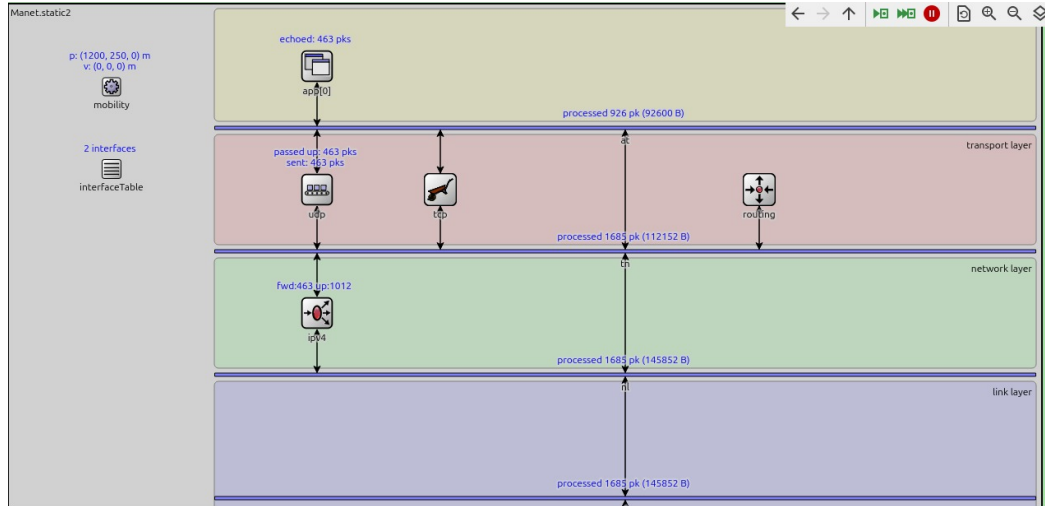


Figura 3.3: Paquetes obtenidos sin caída de nodo en static2 con DSDV

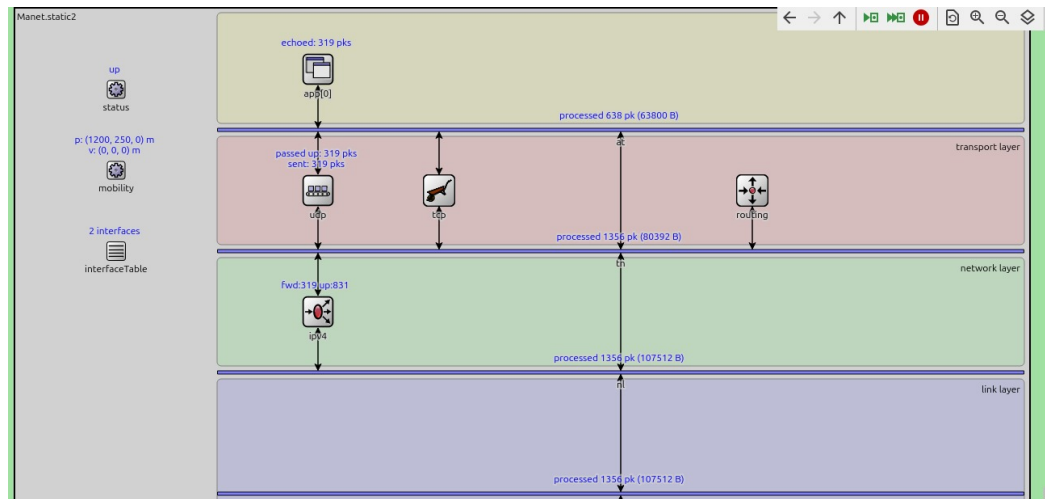


Figura 3.4: Paquetes obtenidos con caída de nodo en static2 con DSDV

En AODV, la pérdida de paquetes es menor porque el protocolo reacciona de forma inmediata ante la rotura de un enlace. Al detectar el fallo, se envía un mensaje RERR hacia el origen, que inicia de forma instantánea un nuevo proceso de descubrimiento de ruta (RREQ), encontrando rápidamente un camino alternativo. En DSDV, la pérdida de paquetes es mayor

debido a su funcionamiento proactivo. La detección del fallo y la actualización de las tablas dependen de mensajes periódicos, por lo que durante el tiempo de convergencia el origen continúa enviando paquetes por una ruta que ya no es válida, lo que provoca una mayor pérdida de datos.

3.4. Con la caída desactivada, simula AODV y DSDV durante 300 s. ¿Qué porcentaje de los `UdpBasicAppData` enviados por `static1` llega a `static2`? Explica los valores y diferencias observadas.

Para este apartado utilizamos las mismas semillas que el apartado anterior.

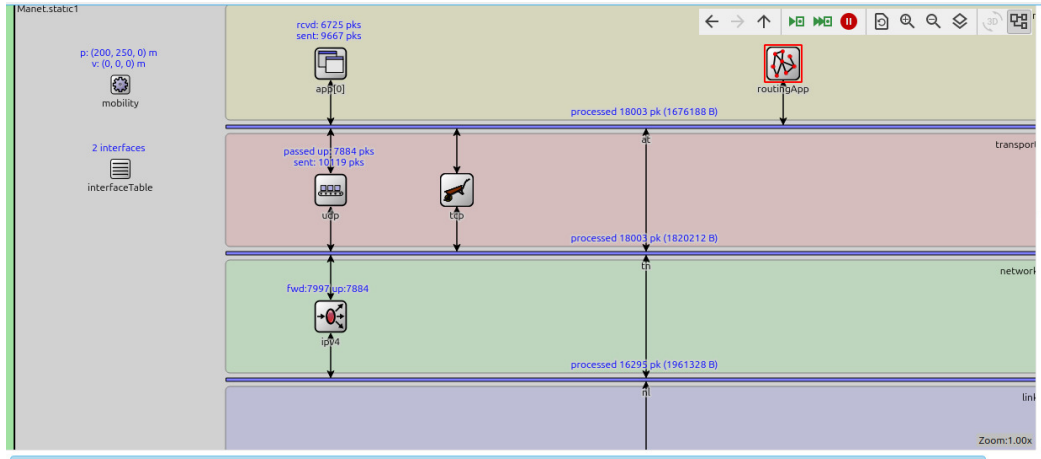


Figura 3.5: Paquetes enviados por `static1` a `static2` en 300 segundos con AODV

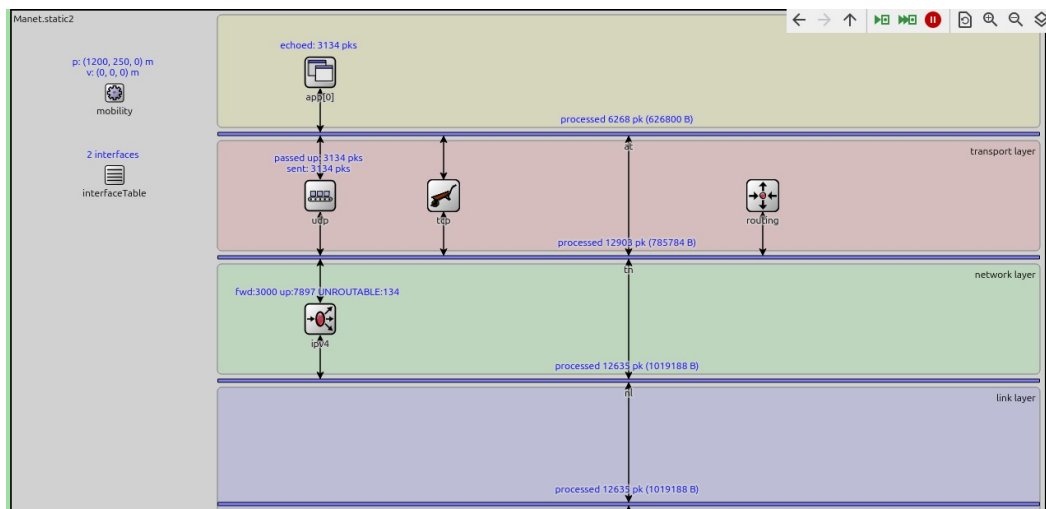


Figura 3.6: Paquetes recibidos por `static2` de `static1` en 300 segundos con AODV

Como se observa en las Figuras 3.5 y 3.6, para AODV el nodo `static1` envía un total de 9667 paquetes a `static2`, de los cuales `static2` recibe 7260 paquetes. Esto supone que llegan correctamente aproximadamente el 75,1% de los paquetes enviados.

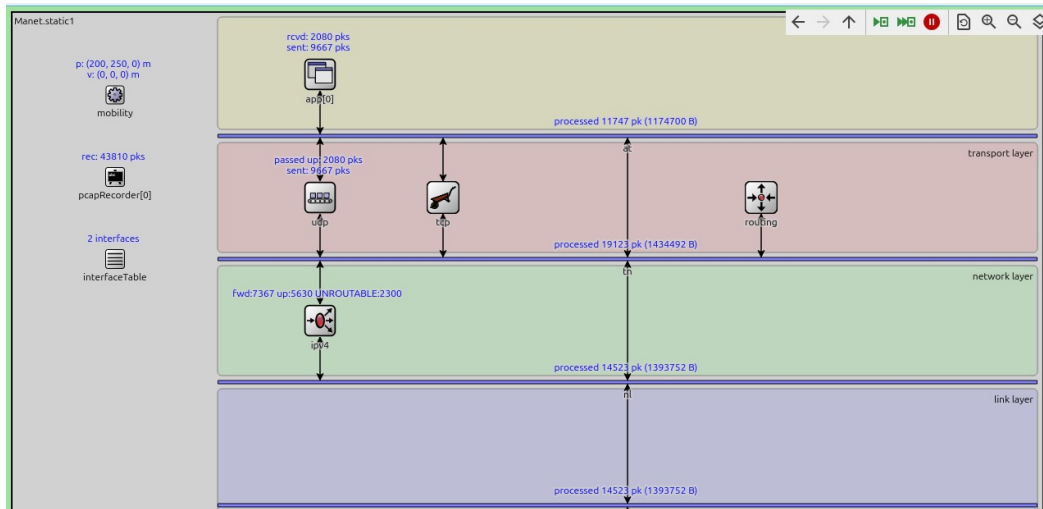


Figura 3.7: Paquetes enviados por static1 a static2 en 300 segundos con DSDV

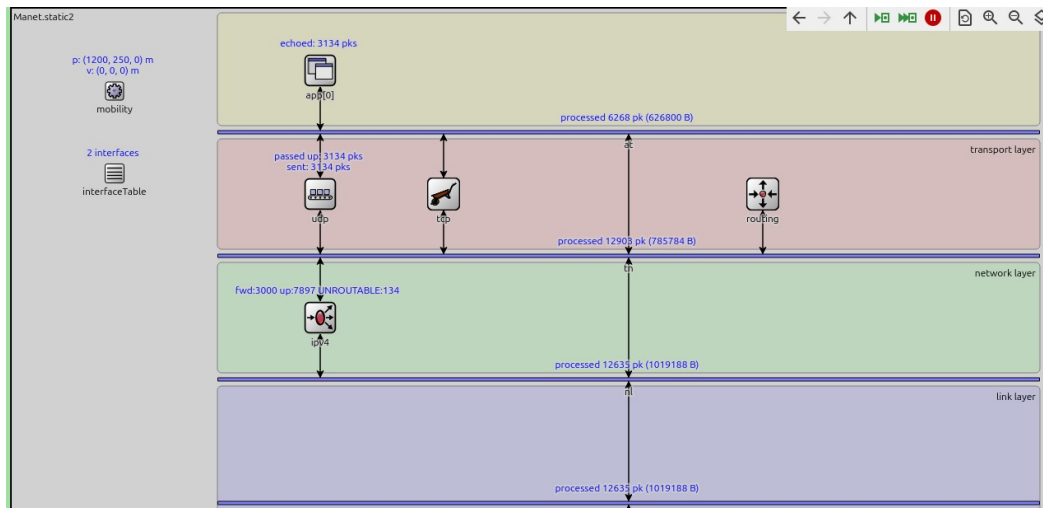


Figura 3.8: Paquetes recibidos por static2 de static1 en 300 segundos con DSDV

Como se observa en las Figuras 3.7 y 3.8, para DSDV, el nodo static1 envía un total de 9667 paquetes a static2, de los cuales static2 recibe 3134 paquetes. Esto supone que llegan correctamente aproximadamente el 32,4% de los paquetes enviados.

En estas simulaciones sin caída de nodos, AODV consigue un porcentaje de entrega claramente superior al de DSDV. En AODV, al tratarse de un protocolo reactivo, las rutas se establecen solo cuando son necesarias y se mantienen actualizadas durante la comunicación, lo que permite una transmisión más eficiente de los paquetes. En cambio, DSDV, al ser un protocolo proactivo, mantiene tablas de enrutamiento mediante actualizaciones periódicas

que generan mayor sobrecarga y pueden provocar el uso de rutas menos eficientes, lo que se traduce en un menor porcentaje de paquetes que llegan correctamente a destino.

3.5. ¿Qué porcentaje de los paquetes devueltos por static2 llegan a static1? Explica los valores y diferencias observadas.

Para AODV, el nodo static2 envía un total de 7260 paquetes, de los cuales static1 recibe 6725, como se observa en las Figuras 3.5 y 3.6. Esto supone que llegan correctamente aproximadamente el 92,6% de los paquetes devueltos. Para DSDV, static2 envía 3131 paquetes y static1 recibe 2080, tal y como se muestra en las Figuras 3.7 y 3.8, lo que equivale a un 66,4% de paquetes recibidos.

La diferencia entre ambos protocolos se debe a su forma de funcionamiento: AODV, al ser reactivo, mantiene rutas activas y actualizadas durante la comunicación, lo que reduce las pérdidas en el envío de respuestas. En cambio, DSDV depende de actualizaciones periódicas de sus tablas de enrutamiento, lo que puede provocar el uso de rutas menos eficientes o desactualizadas, dando lugar a un menor porcentaje de paquetes que llegan correctamente a static1.

AODV vs. DSDV: gráficas

4.1. Obtén una gráfica que muestre el número de paquetes recibidos por static2 en función de la potencia tanto para AODV como para DSDV. Para ello añade al .ini el parámetro repeat para que realice varios experimentos por configuración (al menos 10), modifica seed-1-mt para que genere una semilla diferente para cada experimento y modifica **.radio.transmitter.power para que utilice una potencia distinta para cada ejecución: `$inicio..fin step pasomW`.

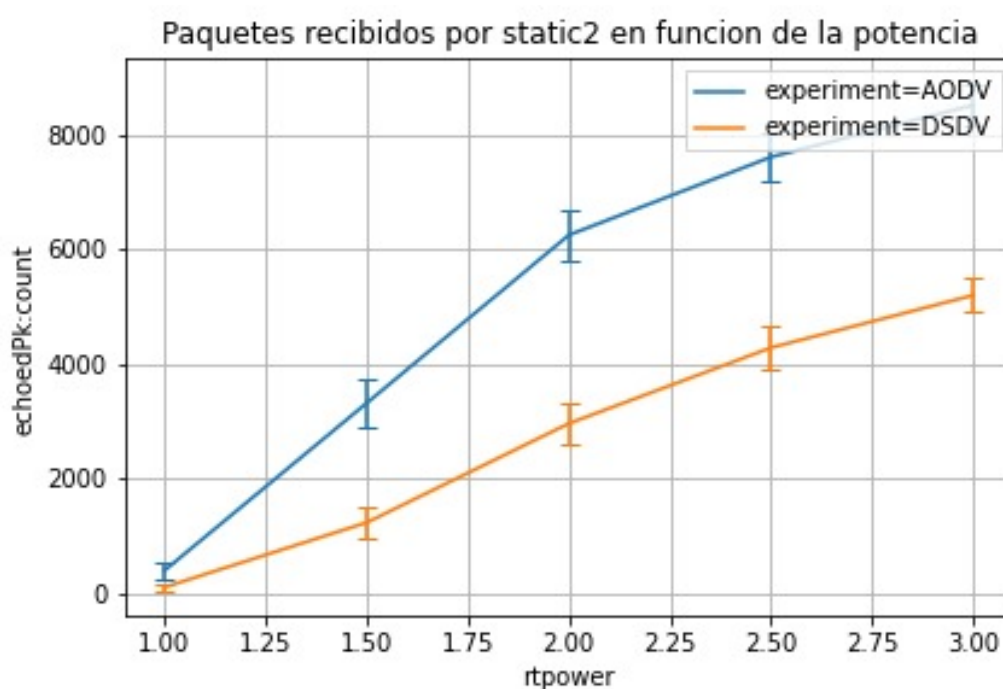


Figura 4.1: Gráfica de paquetes recibidos en static2 según la potencia

Como se muestra en la Figura 4.1, el nodo static2 recibe más paquetes cuando se utiliza AODV que con DSDV a medida que se incrementa la potencia de transmisión. Esto se debe a

la forma de funcionamiento de los protocolos: AODV es reactivo y establece rutas únicamente cuando son necesarias. Al aumentar la potencia, el alcance de los nodos se amplía y surgen más rutas posibles, aunque AODV mantiene la ruta conocida que funciona correctamente, cambiándola solo si se produce un fallo, lo cual es poco frecuente con mayor potencia. Por esta razón, la eficiencia en la entrega de paquetes es mayor con AODV.

4.2. Obtén una gráfica similar para el número de nodos. Explica lo observado.

4.3. Obtén una gráfica como las anteriores para la velocidad. Explica lo observado.

4.4. Para DSDV, obtén una gráfica del porcentaje de paquetes perdidos con diferentes valores de `helloInterval`. Explica lo observado.